

RTL Design Sherpa

Lattice Hardware Architecture Specification 1.2

October 13, 2026

Table of Contents

Lattice Has Index....20

Lattice Hardware Architecture Specification Index....21

Document Organization....22

Front Matter....22

Chapter 1: Introduction....22

Chapter 2: System Overview....22

Chapter 3: Architecture....22

Chapter 4: Interfaces....22

Chapter 5: Performance....22

Chapter 6: Integration....22

Related Documentation....24

Document Information....25

Lattice Hardware Architecture Specification....25

Document Purpose....26

References....27

Terminology....28

Revision History....29

Purpose and Scope....30

Document Purpose....30

Component Overview....31

Document Scope....32

In Scope....32

Out of Scope....32
Specification Level....33
Document Conventions....34
Notation....34
Signal Naming....34
Parameter Naming....34
Diagrams....35
Block Diagrams....35
Address Notation....35
Table Captions....36
Requirement References....37
Definitions and Acronyms....38
Acronyms....38
Terms....39
APB Protocol Signals....40
Use Cases....41
Primary Use Cases....41
UC-1: Simple Peripheral Interconnect (1x4)....41
UC-2: Multi-Master System (2x4)....41
UC-3: Protocol Conversion (1x1)....41
Variant Selection Guide....42
Key Features....43
Feature Summary....43
F1: Parametric MxN Configuration....43

F2: Automatic Address-Based Routing....43
F3: Round-Robin Arbitration....43
F4: Back-to-Back Transactions....43
F5: Proven Building Blocks....44
Feature Comparison....45
Design Philosophy....46
Composition Over Complexity....46
Scalability....46
System Context....47
System Context Diagram....47
Figure 2.1: System Context Diagram....47
Typical Integration Topology....48
Master-Side Connections....48
Slave-Side Connections....48
Interface Boundaries....49
Input Boundary (Master-Side)....49
Output Boundary (Slave-Side)....49
Clock and Reset....49
System Integration Considerations....50
Address Map Planning....50
Multi-Master Considerations....50
Block Diagram....51
Top-Level Architecture....51
Figure 3.1: Lattice Block Diagram....51

Functional Blocks..... 52
Master-Side Protocol Conversion..... 52
Internal Crossbar Fabric..... 52
Slave-Side Protocol Conversion..... 52
Module Hierarchy..... 53
Data Path Width..... 54
Data Flow..... 55
Transaction Flow Overview..... 55
Figure 3.2: Transaction Data Flow..... 55
Write Transaction Flow..... 56
Stage 1: Master Request..... 56
Stage 2: Protocol Conversion (Master-Side)..... 56
Stage 3: Address Decode..... 56
Stage 4: Arbitration (if needed)..... 56
Stage 5: Protocol Conversion (Slave-Side)..... 56
Stage 6: Response Return..... 56
Read Transaction Flow..... 57
Timing Summary..... 58
Back-to-Back Transactions..... 59
Address Mapping..... 60
Address Map Structure..... 60
Figure 3.3: Address Mapping Structure..... 60
Default Address Map..... 61
Address Decode Logic..... 62

Example Decode 62
Customizing Base Address 63
Address Map Constraints 64
Fixed Slave Region Size 64
Maximum Configuration 64
Address Width Requirement 64
Master-Side APB Interfaces 65
Interface Overview 65
Signal Definition 66
APB5 Sideband (mixed-version variants only) 66
Signal Descriptions 68
PSEL (Input) 68
PENABLE (Input) 68
PADDR (Input) 68
PWRITE (Input) 68
PWDATA (Input) 68
PSTRB (Input) 68
PPROT (Input) 68
PRDATA (Output) 68
PSLVERR (Output) 68
PREADY (Output) 69
Transaction Timing 70
Uncontended Write (No Arbitration Wait) 70
Figure 4.1: Uncontended Write at a Master Port 70

Contended Transaction (Arbitration Wait)....70
Figure 4.2: Contended Transfer....71
Slave-Side APB Interfaces....72
Interface Overview....72
Signal Definition....73
APB5 Sideband (mixed-version variants only)....73
Signal Behavior....75
Output Signals (Crossbar to Slave)....75
Input Signals (Slave to Crossbar)....75
Slave Response Handling....76
Wait States....76
Figure 4.3: Slave Wait States....76
Error Response....76
Multi-Slave Considerations....77
Address Overlap Prevention....77
Independent Operation....77
Clock and Reset....78
Clock Interface....78
Signal Definition....78
Clock Requirements....78
Reset Interface....79
Signal Definition....79
Reset Behavior....79
Reset Safe Values....79

Clock-Reset Relationship....80
Reset Assertion....80
Reset Deassertion....80
Figure 4.4: Reset Sequence....80
Integration Notes....81
Clock Domain Crossing....81
Clock Gating....81
Throughput Characteristics....82
Maximum Throughput....82
Single Master....82
Multi-Master (Uncontended)....82
Multi-Master (Contended)....82
Arbitration Handoff....84
Grant Persistence....84
Back-to-Back Timing....84
Throughput Factors....85
Factors That Reduce Throughput....85
Optimal Usage Pattern....85
Latency Analysis....86
Transaction Latency....86
Uncontended Access....86
Contended Access....87
Latency Breakdown....89
Forward Path (Master to Slave)....89

Response Path (Slave to Master)....90
Latency Timing Diagram....91
Best Case (No Waits, No Contention)....91
Figure 5.1: Best-Case Latency Timeline....91
Worst Case (Contention + Slave Wait States)....91
Figure 5.2: Arbitration and Slave Wait States....91
Latency Optimization....92
Design Recommendations....92
Resource Estimates....93
FPGA Resource Usage....93
Pre-Generated Variants....93
Scaling Factors....93
Lines of Code....95
Power Considerations....96
Static Power....96
Dynamic Power....96
Power Optimization....96
Area Comparison....97
Custom Configuration Estimation....98
System Requirements....99
Hardware Requirements....99
Clock and Reset....99
APB Compliance....99
Software Requirements....100

Address Map Configuration....100
Driver Considerations....100
Constraints....101
Address Map Constraints....101
Transaction Constraints....101
Integration Checklist....102
Pre-Integration....102
Post-Integration....102
Parameter Configuration....103
Module Parameters....103
Core Parameters....103
Derived Parameters....104
Parameter Usage Examples....105
Default Configuration....105
Custom Base Address....105
64-bit Data Width....105
Wide Address....105
Parameter Validation....106
Address Width Check....106
Data Width Constraints....106
Custom Generation Parameters....107
Verification Strategy....108
Pre-Verified Component....108
Test Categories....109

Basic Connectivity....109
Address Decode Verification....109
Arbitration Testing....109
Stress Testing....109
APB5 Sideband Gating (mixed variant)....109
Timing and Protocol Compliance....109
Integration Testing Recommendations....110
Level 1: Basic Integration....110
Level 2: Functional Integration....110
Level 3: System Integration....110
Verification Collateral....111
Available Test Infrastructure....111
Running Verification....111
Known Limitations....112
Not Tested....112
Related Documentation....113
Lattice Micro-Architecture Specification Index....114
Document Organization....115
Main Documentation....115
Chapter 1: Architecture....116
Chapter 2: Address Decode and Arbitration....117
Chapter 3: RTL Generator....118
Quick Navigation....119
For New Users....119

For Integration....119
Common Questions....119
Visual Assets....120
Architecture Diagrams....120
Timing Diagrams....120
Component Overview....121
Key Features....121
Pre-Generated Variants....121
Design Philosophy....121
Related Documentation....122
Companion Specifications....122
Project-Level....122
Test Infrastructure....122
RTL....122
Version History....123
Product Requirements Document (PRD)....124
Lattice Generator....124
1. Executive Summary....125
1.1 Quick Stats....125
1.2 Project Goals....125
2. Key Design Principles....126
2.1 Architecture Philosophy....126
2.2 Design Trade-offs....126
3. Architecture Overview....127

3.1 Top-Level Block Diagram..... 127
3.2 Address Mapping..... 128
3.3 Arbitration Strategy..... 129
4. Available Variants..... 131
4.1 Pre-Generated Modules..... 131
4.2 Wrapper Modules..... 131
5. Functional Requirements..... 133
FR-1: Arbitrary MxN Configuration..... 133
FR-2: Round-Robin Arbitration..... 133
FR-3: Address-Based Routing..... 133
FR-4: Back-to-Back Transactions..... 133
FR-5: Configurable Address Map..... 133
6. Interface Specifications..... 134
6.1 Master-Side Interface (APB Slave)..... 134
6.2 Slave-Side Interface (APB Master)..... 134
7. Parameter Configuration..... 135
8. Generator Usage..... 136
8.1 Pre-Generated Variants..... 136
8.2 Custom Generation..... 136
8.3 Generator Options..... 136
9. Performance Characteristics..... 137
9.1 Latency..... 137
9.2 Throughput..... 137
9.3 Resource Utilization (Estimated)..... 137

10. Verification Status..... 138
10.1 Test Coverage..... 138
10.2 Test Methodology..... 138
11. Integration Guide..... 139
11.1 Simple Integration (1to4)..... 139
11.2 Multi-Master Integration (2to4)..... 139
12. Design Constraints..... 141
12.1 Limitations..... 141
12.2 Assumptions..... 141
13. Future Enhancements..... 142
13.1 Potential Features..... 142
13.2 Generator Improvements..... 142
14. References..... 143
14.1 Internal Documentation..... 143
14.2 External Standards..... 143
14.3 Related Components..... 143
Claude Code Guide: Lattice Component..... 144
Quick Context..... 145
Critical Rules for This Component..... 146
Rule #0: This is a GENERATOR, Not Just Modules..... 146
Rule #1: Address Map is Fixed Per-Slave..... 146
Rule #2: Know the Pre-Generated Variants..... 146
Rule #3: Generator Syntax..... 147
Architecture Quick Reference..... 148

Module Organization....148
Block Diagram (2×4 Example)....148
Common User Questions and Responses....150
Q: “How do I connect a CPU to 4 peripherals?”....150
Q: “I need CPU and DMA to access peripherals. Which crossbar?”....150
Q: “What if I need 3 masters and 8 slaves?”....151
Q: “How does arbitration work?”....151
Q: “Can I change the address map?”....152
Q: “How do I run tests?”....152
Integration Patterns....154
Pattern 1: Simple Peripheral Interconnect....154
Pattern 2: Multi-Master System....154
Pattern 3: Hierarchical Interconnect....155
Anti-Patterns to Catch....156
Anti-Pattern 1: Generating When Pre-Generated Exists....156
Anti-Pattern 2: Assuming Custom Per-Slave Sizes....156
Anti-Pattern 3: Not Mentioning Address Map....156
Anti-Pattern 4: Forgetting About Wrappers....156
Debugging Workflow....158
Issue: Address Not Routing Correctly....158
Issue: Arbitration Not Fair....158
Issue: Back-to-Back Transactions Stalling....158
Quick Commands....159
Remember....160

List of Figures

Figure 2.1: System Context Diagram....46
Figure 3.1: Lattice Block Diagram....50
Figure 3.2: Transaction Data Flow....54
Figure 3.3: Address Mapping Structure....59
Figure 4.1: Uncontended Write at a Master Port....69
Figure 4.2: Contended Transfer....70
Figure 4.3: Slave Wait States....75
Figure 4.4: Reset Sequence....79
Figure 5.1: Best-Case Latency Timeline....89
Figure 5.2: Arbitration and Slave Wait States....89

List of Tables

Table 1: Reference Documents....26
Table 2: Terminology and Definitions....27
Table 3: Document Revision History....28
Table 4: HAS vs MAS Coverage Comparison....32
Table 5: Signal Naming Conventions....33
Table 6: Parameter Naming Conventions....33
Table 7: Acronym Definitions....37
Table 8: Term Definitions....38
Table 9: APB Protocol Signal Definitions....39
Table 10: Variant Selection Guide....41
Table 11: Pre-Generated Variant Comparison....44
Table 12: Typical Master Connections....47
Table 13: Typical Slave Connections (1to4 example)....47
Table 14: Data Path Width Parameters....53
Table 15: Transaction Timing Summary....57
Table 16: Default Address Map (4-slave example)....60
Table 17: Address Decode Example....61
Table 18: Master Port Signal Definition....65
Table 19: APB5 Master Sideband Signals....65
Table 20: Slave Port Signal Definition....72
Table 21: APB5 Slave Sideband Signals....72
Table 22: Clock Signal....77
Table 23: Reset Signal....78
Table 24: Reset Safe Values....78
Table 25: Single-Master-Active Throughput, by Variant Class....81
Table 26: Contended Access Throughput....82
Table 27: Throughput Reduction Factors....84
Table 28: Uncontended Transaction Latency....85
Table 29: Measured Timing, Always-Ready Slave....85
Table 30: Arbitration Latency....86
Table 31: Forward Path Latency....87
Table 32: Response Path Latency....88
Table 33: Latency Optimization Recommendations....90
Table 34: Resource Scaling Factors....91
Table 35: Source Code Metrics....93
Table 36: Power Optimization Techniques....94

Table 37: Relative Area Comparison....95
Table 38: Clock and Reset Requirements....97
Table 39: APB Compliance Requirements....97
Table 40: Driver Considerations....98
Table 41: Address Map Constraints....99
Table 42: Transaction Constraints....99
Table 43: Core Parameter Definition....101
Table 44: Derived Parameters....102
Table 45: Valid Data Width Configurations....104
Table 46: Generator Command Line Options....105
Table 47: Level 1 Integration Tests....108
Table 48: Level 2 Integration Tests....108
Table 49: Level 3 Integration Tests....108
Table 50: Test Infrastructure....109
Table 51: Known Limitations....110

Lattice Has Index

Generated: 2026-10-13

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

Lattice Hardware Architecture Specification Index

Version: 1.2 **Date:** 2026-10-12 **Purpose:** High-level hardware architecture specification for Lattice component

---

Document Organization

Note: All chapters are linked below; this index doubles as the manifest for automated document generation.

Front Matter

- Document Information

Chapter 1: Introduction

- Purpose and Scope
- Document Conventions
- Definitions and Acronyms

Chapter 2: System Overview

- Use Cases
- Key Features
- System Context

Chapter 3: Architecture

- Block Diagram
- Data Flow
- Address Mapping

Chapter 4: Interfaces

- Master-Side APB Interfaces
- Slave-Side APB Interfaces
- Clock and Reset

Chapter 5: Performance

- Throughput Characteristics
- Latency Analysis
- Resource Estimates

Chapter 6: Integration

- System Requirements
- Parameter Configuration
- Verification Strategy

Related Documentation

- **Lattice MAS** - Micro-Architecture Specification (detailed block-level)
- **PRD.md** - Product requirements and overview
- **CLAUDE.md** - AI development guide

Last Updated: 2026-10-12 **Maintained By:** RTL Design Sherpa Project

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

Document Information

Lattice Hardware Architecture Specification

Document Number: LATTICE-HAS-001 **Version:** 1.2 **Status:** Production Ready **Classification:** Open Source - MIT License (matches the SPDX tags on every RTL file)

Document Purpose

This Hardware Architecture Specification (HAS) is the high-level architectural view of the Lattice: system-level design, external interfaces, performance characteristics, and integration requirements. Internal implementation specifics are deliberately left to the MAS.

Target Audience: - System architects evaluating Lattice for SoC integration - Hardware engineers planning peripheral interconnect design - Software engineers developing drivers and firmware - Verification engineers planning system-level testing

Companion Documents: - Lattice Micro-Architecture Specification (MAS) - Detailed block-level implementation - Lattice Product Requirements Document (PRD) - Requirements and rationale

---

References

Reference Documents

ID	Document	Description
[REF-1]	Lattice MAS v1.2	Micro-Architecture Specification
[REF-2]	Lattice PRD	Product Requirements Document
[REF-3]	ARM AMBA APB	APB Protocol Specification (IHI 0024C)

Terminology

Terminology and Definitions

Term	Definition
APB	Advanced Peripheral Bus - ARM AMBA low-power peripheral bus
Arbiter	Logic that selects between competing requests using round-robin
Crossbar	NxM interconnect fabric connecting multiple masters to multiple slaves
Master	APB initiator device that initiates read/write transactions
PSEL	APB peripheral select signal indicating slave selection
PENABLE	APB enable signal indicating data phase of transaction
PREADY	APB ready signal indicating slave response completion
Slave	APB target device that responds to master transactions
Transaction	Complete APB read or write operation (setup + data phase)

Revision History

Document Revision History

Version	Date	Author	Description
1.0	2026-01-03	seang	Initial HAS release
1.1	2026-10-12	seang	Refresh: update LOC counts, verification status, and generator CLI options
1.2	2026-10-13	seang	Rebrand: component codename **lattice**, directory `lattice-apbx-xbar`; book titles and document number (LATTICE-HAS-001) updated; RTL module names unchanged (`apbx_xbar_*`)

Last Updated: 2026-10-13

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

Purpose and Scope

Document Purpose

This Hardware Architecture Specification (HAS) describes the Lattice at the system integration level, with enough detail for:

- **System architects** to evaluate the component for SoC integration
- **Hardware engineers** to plan interconnect topology and address mapping
- **Software engineers** to understand register access paths and address spaces
- **Verification engineers** to develop system-level test plans

Component Overview

The Lattice is a parametric MxN interconnect fabric connecting multiple APB masters to multiple APB slaves:

- **Automatic address-based routing** to the appropriate slave
- **Fair round-robin arbitration** when multiple masters access the same slave
- **Back-to-back transactions** with no master-side idle cycles (they do not overlap inside the fabric – 10 pclk cycles each PREADY-to-PREADY with one master, 11 on an arbitrated variant, where the round-robin grant is registered; see 5.1)
- **Scalable configuration** from 1x1 to 16x16

Document Scope

In Scope

- System-level architecture and block organization
- External interface specifications (all APB signals)
- Address mapping and routing strategy
- Arbitration policy and fairness guarantees
- Performance characteristics (latency, throughput)
- Integration requirements and parameters

Out of Scope

- Internal RTL implementation details (see MAS)
- Detailed timing diagrams for internal signals
- Synthesis and implementation guidance
- Test infrastructure details

Specification Level

This HAS is a **black-box** view of the Lattice:

HAS vs MAS Coverage Comparison

Aspect	HAS Coverage	MAS Coverage
External ports	Complete	Complete
Functional behavior	Complete	Complete
Internal architecture	Overview only	Detailed
FSM states	-	Complete
Timing paths	Interface only	All paths
RTL structure	-	Complete

Next: Document Conventions

Document Conventions

Notation

Signal Naming

Signal Naming Conventions

Convention	Meaning	Example
`m<i>_apb_*`	Master-side APB port *i* – BOTH directions: requests in (`PSEL`, `PADDR`, ...), responses out (`PRDATA`, `PREADY`, `PSLVERR`)	`m0_apb_PSEL` (in), `m0_apb_PREADY` (out)
`s<j>_apb_*`	Slave-side APB port *j* – BOTH directions: requests out, responses in	`s2_apb_PSEL` (out), `s2_apb_PREADY` (in)
`P*`	APB protocol signal prefix	`PADDR`, `PWRITE`, `PRDATA`
`<i>`, `<j>`	Master/slave index placeholder	m0, s3

Parameter Naming

Parameter Naming Conventions

Convention	Meaning	Example
`UPPER_CASE`	Module parameters	`ADDR_WIDTH`, `DATA_WIDTH`
`NUM_*`	Count parameters	`NUM_MASTERS`, `NUM_SLAVES`
`*_WIDTH`	Bit width parameters	`DATA_WIDTH = 32`
`BASE_*`	Base address parameters	`BASE_ADDR = 0x10000000`

Diagrams

Block Diagrams

Block diagrams here use:

- **Rectangles** for functional blocks
- **Arrows** showing data flow direction
- **Dashed lines** for configuration/control paths
- **Labels** on connections indicating signal groups

Address Notation

Addresses appear in hexadecimal with the `0x` prefix:

- `0x1000_0000` - Underscores separate 16-bit groups for readability
- `[31:0]` - Bit range notation (MSB:LSB)
- `64KB` - Size notation using standard abbreviations

Table Captions

Tables are numbered sequentially within each chapter. A colon (`:`) followed by the caption text appears below each table.

Requirement References

Requirements from the PRD are referenced as:

- **FR-n** - Functional Requirement number n
- **PR-n** - Performance Requirement number n
- **IR-n** - Interface Requirement number n

Next: Definitions and Acronyms

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

Definitions and Acronyms

Acronyms

Acronym Definitions

Acronym	Definition
AMBA	Advanced Microcontroller Bus Architecture
APB	Advanced Peripheral Bus
CPU	Central Processing Unit
DMA	Direct Memory Access
FSM	Finite State Machine
GPIO	General Purpose Input/Output
HAS	Hardware Architecture Specification
I2C	Inter-Integrated Circuit
LOC	Lines of Code
LUT	Lookup Table (FPGA resource)
MAS	Micro-Architecture Specification
PRD	Product Requirements Document
RTL	Register Transfer Level
SoC	System on Chip
SPI	Serial Peripheral Interface
UART	Universal Asynchronous Receiver/Transmitter

Terms

Term Definitions

Term	Definition
Address Decode	Logic that determines which slave should receive a transaction based on address
Arbitration	Process of selecting one request when multiple masters compete for the same slave
Back-to-back	Consecutive transactions with no idle cycles between them
Crossbar	Interconnect topology allowing any master to communicate with any slave
Grant Persistence	Maintaining arbitration grant through entire transaction duration
Master	Device that initiates APB transactions (CPU, DMA, etc.)
Round-Robin	Arbitration scheme that rotates priority among requesters
Slave	Device that responds to APB transactions (peripherals)
Starvation	Condition where a requester is indefinitely denied access
Transaction	Complete APB read or write operation including setup and data phases
Back-to-Back	Consecutive transactions with no master-side idle cycles. Note this crossbar does not OVERLAP them, and cadence is one cycle longer than single-transfer latency: 10 PREADY-to-PREADY against 9 SETUP-to-PREADY with one master, 11 against 10 when arbitrated (see 5.2)

APB Protocol Signals

APB Protocol Signal Definitions

Signal	Direction	Description
PCLK	Input	APB clock (all signals synchronous to rising edge)
PRESETn	Input	Active-low reset. Assertion is asynchronous in every build; deassertion must be synchronised to PCLK (see 4.3)
PADDR	Master to Slave	Address bus
PSEL	Master to Slave	Peripheral select (one per slave)
PENABLE	Master to Slave	Enable signal (high during data phase)
PWRITE	Master to Slave	Write/read direction (1=write, 0=read)
PWDATA	Master to Slave	Write data bus
PSTRB	Master to Slave	Write strobes (byte enables)
PPROT	Master to Slave	Protection attributes
PRDATA	Slave to Master	Read data bus
PREADY	Slave to Master	Ready signal (can extend transaction)
PSLVERR	Slave to Master	Slave error response

Next: Chapter 2: System Overview

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

Use Cases

Primary Use Cases

UC-1: Simple Peripheral Interconnect (1x4)

Scenario: Single CPU accessing multiple peripherals

A microcontroller reaches UART, GPIO, Timer, and SPI peripherals through a single APB master interface.

Configuration: - 1 Master (CPU) - 4 Slaves (UART, GPIO, Timer, SPI) - Variant: `apbx_xbar_1to4`

Address Map:

```
0x1000_0000 - 0x1000_FFFF : UART  (64KB)
0x1001_0000 - 0x1001_FFFF : GPIO  (64KB)
0x1002_0000 - 0x1002_FFFF : Timer (64KB)
0x1003_0000 - 0x1003_FFFF : SPI   (64KB)
```

UC-2: Multi-Master System (2x4)

Scenario: CPU and DMA both accessing peripherals

An SoC where both the CPU and a DMA controller share the same peripherals.

Configuration: - 2 Masters (CPU, DMA) - 4 Slaves (peripherals) - Variant: `apbx_xbar_2to4`

Key Behavior: - Round-robin arbitration when both masters access same peripheral - No master starvation guaranteed - Back-to-back transactions on uncontended access (10 pclk cycles each for M = 1, 11 when arbitrated; they do not overlap – see 5.1)

UC-3: Protocol Conversion (1x1)

Scenario: APB timing boundary or clock domain preparation

Insert the APB crossbar as a timing stage, or as preparation for clock domain crossing.

Configuration: - 1 Master - 1 Slave - Variant: `apbx_xbar_1to1`

Benefits: - Isolates master from slave timing - Consistent transaction buffering - Lowest resource footprint of the family

Variant Selection Guide

Variant Selection Guide

Use Case	Recommended Variant	Rationale
Simple peripheral bus	`apbx_xbar_1to4`	Most common SoC configuration
CPU + DMA system	`apbx_xbar_2to4`	Fair access for both masters
Timing isolation	`apbx_xbar_1to1`	Registered command and response paths on a single pairing
Multi-master arbitration	`apbx_xbar_2to1`	Focus on arbitration only
Custom topology	Generator	Any MxN configuration

Next: Key Features

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

Key Features

Feature Summary

F1: Parametric MxN Configuration

- Any combination of M masters and N slaves
- Pre-generated variants for common configurations (1x1, 2x1, 1x4, 2x4, and the mixed-version 2x2)
- Python generator for custom configurations up to 16x16
- A single GENERATOR serves all configurations – not a single RTL source. Each variant is a distinct file with its M×N baked into the module name and port list; only ADDR_WIDTH, DATA_WIDTH, STRB_WIDTH and BASE_ADDR are elaboration parameters (see 6.2). The parameterized-core reading was true of the retired `apbx_xbar_thin` and has not been true since 2026-08-27.

F2: Automatic Address-Based Routing

- Parallel address decode for all slaves
- Fixed 64KB address space per slave
- Configurable base address via `BASE_ADDR` parameter
- Zero-decode-latency routing

Address Calculation:

```
slave_index = ((PADDR - BASE_ADDR) >> 16)[$clog2(S)-1:0]
```

F3: Round-Robin Arbitration

- Independent arbiter per slave
- Fair rotation of master priority
- No master starvation guaranteed
- Grant persistence through transaction completion

F4: Back-to-Back Transactions

- Accepted with no master-side idle cycles required
- Grant held for the duration of one transaction, released at the response handshake
- They do NOT overlap inside the fabric: `apb4_slave` is one-command-at-a-time. Sustained cadence measures 10 pclk cycles PREADY-to-PREADY, one

more than the 9-cycle SETUP-to-PREADY single transfer, the extra cycle being the next transfer's mandatory SETUP phase (see 5.1/5.2)

F5: Proven Building Blocks

- Built from production-tested `apb4_slave.sv` and `apb4_master.sv` modules
- No new protocol logic - pure composition
- Each component independently verified

Feature Comparison

Pre-Generated Variant Comparison

Feature	apbx_xbar_1to1	apbx_xbar_2to1	apbx_xbar_1to4	apbx_xbar_2to4	apbx_xbar_2to2_mixed
Masters	1	2	1	2	2
Slaves	1	1	4	4	2
Arbitration	No	Yes	No	Yes	Yes
Address Decode	No	No	Yes	Yes	Yes
APB5 sideband	No	No	No	No	Yes – per-port version gating

All five are generator output. LOC per variant is in HAS ch05_performance/03_resources.md, the single source of truth for it – the row that used to sit in this table drifted stale within two days of being reconciled. `apbx_xbar_thin`, a hand-written parameterized core that once occupied a sixth column here, was retired and deleted on 2026-08-27 along with its tests, formal harnesses and testplan; nothing in the tree depends on it.

Design Philosophy

Composition Over Complexity

The crossbar is a composition of well-tested building blocks:

1. **APB Slaves** (M instances) - Convert incoming APB to internal cmd/rsp
2. **Arbiters** (N instances) - Select winning master per slave
3. **Address Decode** - Route commands to appropriate slave
4. **Response Routing** - Return responses to originating masters
5. **APB Masters** (N instances) - Convert internal cmd/rsp back to APB

Scalability

Resource usage scales predictably: - M x N routing paths - N independent arbiters - Linear growth with configuration size

---

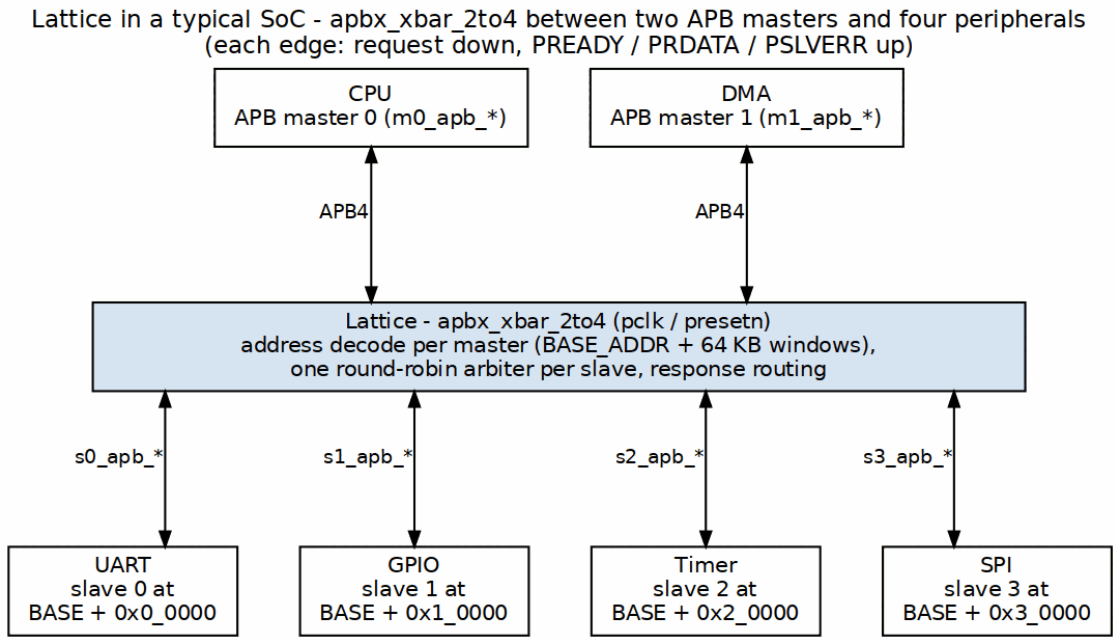
Next: System Context

System Context

System Context Diagram

The Lattice in a typical SoC context:

Figure 2.1: System Context Diagram



System Context Diagram

Typical Integration Topology

Master-Side Connections

The crossbar's master ports face upstream APB masters:

Typical Master Connections

Master Type	Typical Source	Connection Point
CPU	AHB-to-APB Bridge	`m0_apb_*`
DMA	DMA Controller APB Port	`m1_apb_*`
Debug	Debug Access Port	`m2_apb_*` (if present)

Slave-Side Connections

The crossbar's slave ports face downstream APB peripherals:

Typical Slave Connections (1to4 example)

Slave Type	Address Offset	Connection Point
UART	+0x0_0000	`s0_apb_*`
GPIO	+0x1_0000	`s1_apb_*`
Timer	+0x2_0000	`s2_apb_*`
SPI	+0x3_0000	`s3_apb_*`

Interface Boundaries

Input Boundary (Master-Side)

Each master port presents a complete APB slave interface:

- Receives PSEL, PENABLE, PADDR, PWRITE, PWDATA, PSTRB, PPROT
- Returns PRDATA, PSLVERR, PREADY

Output Boundary (Slave-Side)

Each slave port presents a complete APB master interface:

- Drives PSEL, PENABLE, PADDR, PWRITE, PWDATA, PSTRB, PPROT
- Receives PRDATA, PSLVERR, PREADY

Clock and Reset

Single clock domain: - `pclk` - APB clock (all signals synchronous) - `presetn` - Active-low asynchronous reset

System Integration Considerations

Address Map Planning

The fixed 64KB per-slave allocation requires planning:

1. **Peripheral fit:** Ensure each peripheral needs <= 64KB address space
2. **Base address:** Set `BASE_ADDR` to avoid conflicts with other subsystems
3. **Address width:** `ADDR_WIDTH` must accommodate BASE_ADDR + (NUM_SLAVES x 64KB)

Multi-Master Considerations

When using multiple masters:

1. **Arbitration overhead:** a contended access waits a FULL transaction, not 1-2 cycles. The grant is held to the response handshake (`WAIT_GNT_ACK(1)`), so each master ahead in the round-robin costs ~11 cycles (the arbitrated back-to-back period). Uncontended, the registered grant still costs exactly 1 cycle – see HAS 5.2.
2. **Priority:** All masters have equal priority (round-robin)
3. **Bandwidth:** Shared access reduces per-master bandwidth

Next: Chapter 3: Architecture

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

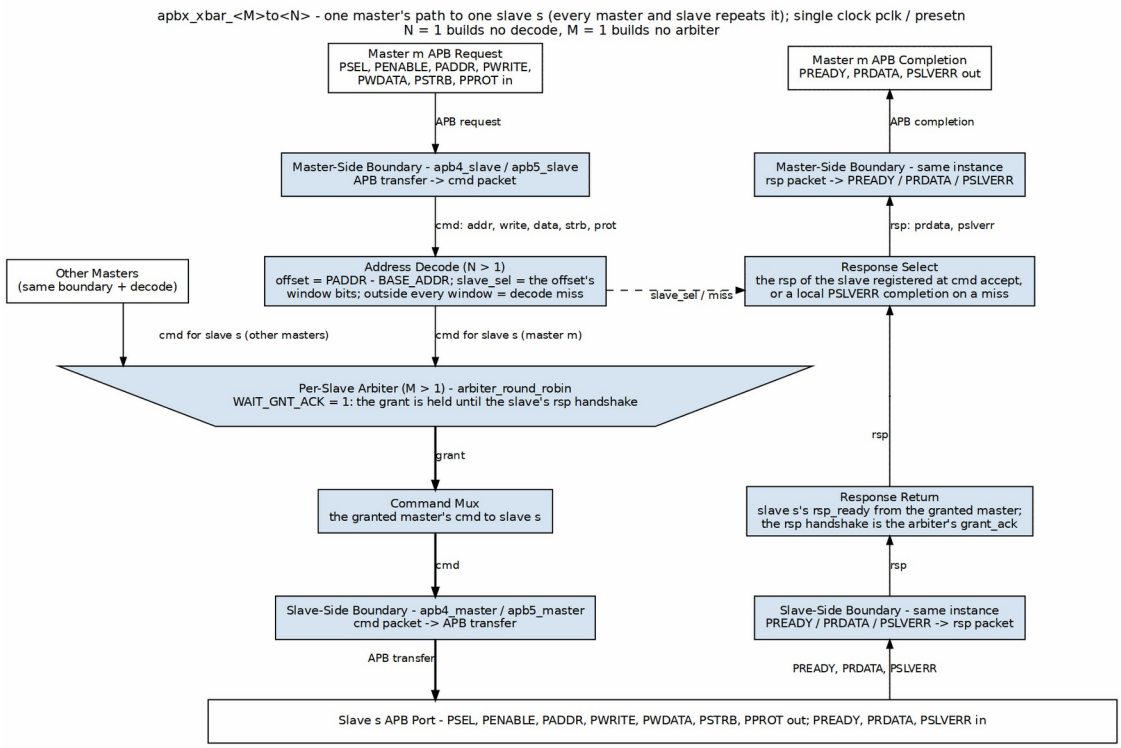
---

Block Diagram

Top-Level Architecture

The Lattice has three functional layers:

Figure 3.1: Lattice Block Diagram



Block Diagram

One master's path to one slave; every master and slave repeats it. The request takes the center column: the master-side `apb4_slave` turns the APB transfer into a cmd packet, the address decode picks the slave window, and the slave's round-robin arbiter (the trapezium, where every master's cmd for that slave contends) grants one master and holds the grant until the response handshake. The completion climbs the right column, routed back by the slave index registered when the command was accepted; an address outside every window completes locally with PSLVERR.

Functional Blocks

Master-Side Protocol Conversion

Component: `apb4_slave[M]` instances (one per master)

Each APB slave instance: - Accepts incoming APB transactions from a master - Converts the APB protocol to the internal command/response format - Buffers the transaction - Generates the response timing

Interface: - Input: Complete APB slave signals from external master - Output: Internal cmd/rsp bus to crossbar fabric

Internal Crossbar Fabric

Components: - **Address Decode** - Parallel decode logic - **Per-Slave Arbiters** - Round-robin arbiter per slave (N instances) - **Response Routing** - Combinational response mux (the registers on this path live in the boundary IP's skid buffers, not the mux)

The crossbar fabric: 1. Decodes transaction address to determine target slave 2. Arbitrates when multiple masters request the same slave 3. Routes commands to the selected slave 4. Routes responses back to the originating master

Slave-Side Protocol Conversion

Component: `apb4_master[N]` instances (one per slave)

Each APB master instance: - Accepts internal cmd/rsp from crossbar fabric - Generates compliant APB transactions to the slave - Handles wait states via PREADY - Propagates error responses via PSLVERR

Interface: - Input: Internal cmd/rsp bus from crossbar fabric - Output: Complete APB master signals to external slave

Module Hierarchy

```
apbx_xbar_MxN
+-- apb4_slave[0..M-1]          # Master-side conversion
+-- addr_decode                 # Parallel address decode
+-- arbiter[0..N-1]            # Per-slave round-robin
+-- response_mux               # Response routing
+-- apb4_master[0..N-1]         # Slave-side conversion
```

Data Path Width

Data path widths are consistent throughout:

Data Path Width Parameters

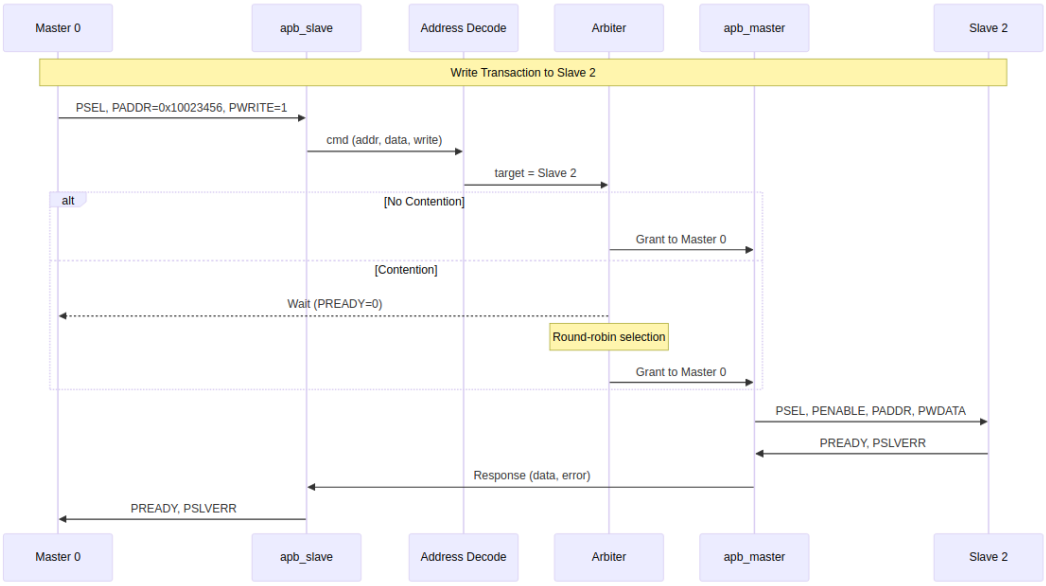
Parameter	Default	Range	Description
ADDR_WIDTH	32	18-64 (decoding variants: the slave-index part-select reads bit 17); 1-64 for 1to1/2to1	Address bus width
DATA_WIDTH	32	8-64	Data bus width
STRB_WIDTH	4	1-8	Byte strobe width (DATA_WIDTH/8)

Next: Data Flow

Data Flow

Transaction Flow Overview

Figure 3.2: Transaction Data Flow



Data Flow Diagram

Write Transaction Flow

A write transaction passes through six stages:

Stage 1: Master Request

The external master initiates an APB write: 1. Master asserts PSEL and PADDR with target address 2. Master sets PWRITE=1 and provides PWDATA 3. Next cycle: Master asserts PENABLE

Stage 2: Protocol Conversion (Master-Side)

The `apb4_slave` module converts to internal format: 1. Captures transaction parameters (address, data, write) 2. Generates internal command 3. Routes to address decode

Stage 3: Address Decode

Parallel decode determines target slave:

```
slave_index = offset[16 +: $clog2(S)]      // offset = PADDR -
BASE_ADDR
```

A part-select, not a mask. `& (S-1)` appeared here until 2026-08-30 and is only equivalent when S is a power of two – for a generated 3-slave variant the mask would make slave 1 unreachable. The generator emits a part-select (`slave_sel = m0_cmd_offset[13:12]` for S = 3 at the generator's default 4KB window; the shipped 64KB variants read `offset[16 +: $clog2(S)]`), and the sibling formulas in ch02/ch03 always did.

Stage 4: Arbitration (if needed)

If multiple masters target the same slave: 1. Arbiter selects one master (round-robin priority) 2. Other masters wait (PREADY low) 3. Grant held until transaction completes

Stage 5: Protocol Conversion (Slave-Side)

The `apb4_master` module generates the APB transaction: 1. Asserts PSEL to selected slave 2. Provides PADDR, PWRITE, PWDATA 3. Asserts PENABLE next cycle 4. Waits for slave PREADY

Stage 6: Response Return

The response flows back to the originating master: 1. Slave asserts PREADY (with optional PSLVERR) 2. `apb4_master` captures response 3. Response routed to correct `apb4_slave` 4. `apb4_slave` asserts PREADY to original master

Read Transaction Flow

Read transactions follow the same flow with these differences:

- PWRITE=0 indicates read operation
- No PWDATA on forward path
- PRDATA returned on response path

Timing Summary

Transaction Timing Summary

Transaction Phase	Clock Cycles	Notes
Setup phase	1	PSEL asserted, PENABLE low
Data phase	1+	PENABLE high, wait for PREADY
Decode	0	Combinational (parallel decode)
Arbitration	0 or 1	0 when M = 1 – the generator emits no arbiter at all. 1 when M > 1, even uncontended, because the grant is REGISTERED

Back-to-Back Transactions

The crossbar accepts back-to-back transactions – but they do NOT overlap:

1. A master may hold PSEL and start the next SETUP the cycle after PREADY
2. `apb4_slave` is one-command-at-a-time, so the next command is not captured until the previous transaction completes
3. Sustained cadence is therefore one cycle LONGER than single-transfer latency: measured PREADY-to-PREADY = 10 pclk cycles at the earliest legal turnaround, against 9 cycles SETUP-to-PREADY for one transfer (M = 1; 11 and 10 on an arbitrated variant). The extra cycle is the next transfer's mandatory SETUP phase, which cannot overlap the previous ACCESS (see 5.1/5.2)

Next: Address Mapping

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

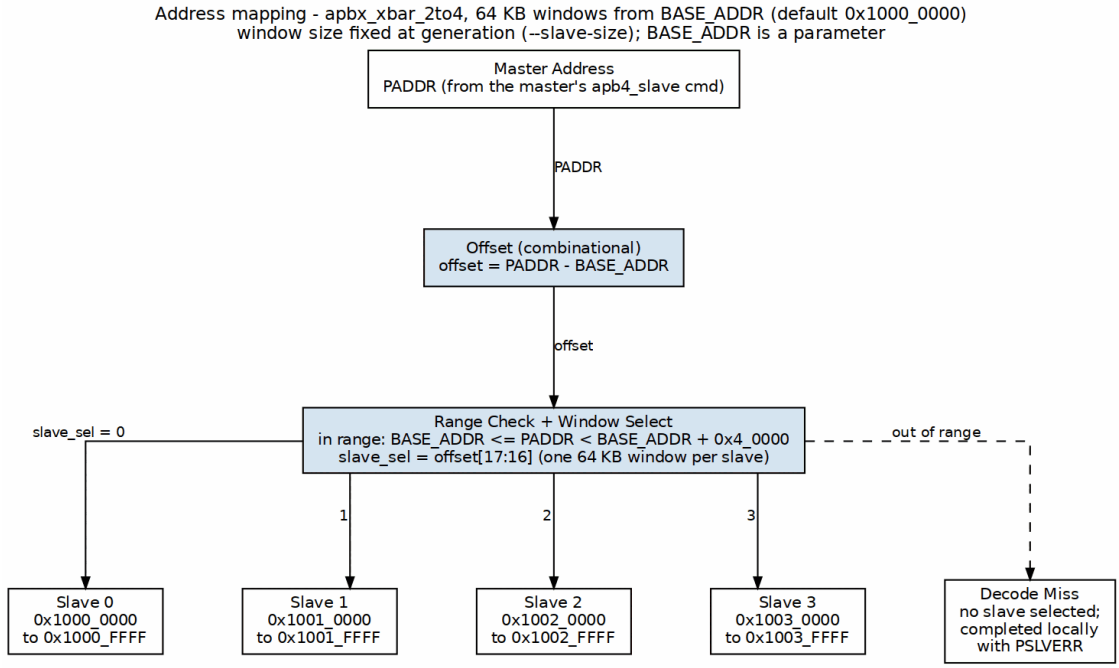
---

Address Mapping

Address Map Structure

The shipped Lattice variants use a 64KB address region per slave. The size is chosen at generation (`--slave-size`, generator default 4KB) and then baked into the decode part-select, so it is fixed for a given generated module:

Figure 3.3: Address Mapping Structure



Address Mapping

Default Address Map

With default `BASE_ADDR = 0x1000_0000`:

Default Address Map (4-slave example)

Slave Index	Start Address	End Address	Size
0	0x1000_0000	0x1000_FFFF	64KB
1	0x1001_0000	0x1001_FFFF	64KB
2	0x1002_0000	0x1002_FFFF	64KB
3	0x1003_0000	0x1003_FFFF	64KB
…	…	…	…
N-1	BASE + (N-1)*64KB	BASE + N*64KB - 1	64KB

Address Decode Logic

The target slave comes from simple bit extraction:

```
offset = PADDR - BASE_ADDR
slave_index = offset[16 +: $clog2(S)]   // ceil(log2(S)) bits: [17:16]
for S=4
local_addr = offset[15:0]      // Bits 15:0 are local address within
slave
```

Example Decode

For address `0x1002_3456` with `BASE_ADDR = 0x1000_0000`:

Address Decode Example

Step	Calculation	Result
1. Compute offset	0x1002_3456 - 0x1000_0000	0x0002_3456
2. Extract slave index	offset[17:16] (S=4)	0x2 (Slave 2)
3. Extract local address	offset[15:0]	0x3456

Customizing Base Address

The BASE_ADDR parameter shifts the entire address map:

```
// Peripheral space at 0x4000_0000
apbx_xbar_1to4 #(
    .BASE_ADDR(32'h4000_0000)
) u_xbar (...);

// Resulting map:
// Slave 0: 0x4000_0000 - 0x4000_FFFF
// Slave 1: 0x4001_0000 - 0x4001_FFFF
// Slave 2: 0x4002_0000 - 0x4002_FFFF
// Slave 3: 0x4003_0000 - 0x4003_FFFF
```

Address Map Constraints

Fixed Slave Region Size

Each slave occupies exactly 64KB (0x10000 bytes). This is a current design constraint.

Implications: - Peripherals requiring more than 64KB need multiple slave ports - Peripherals using less than 64KB have unused address space

Maximum Configuration

With 4-bit slave index extraction: - Maximum 16 slaves - Total address space: 16 x 64KB = 1MB

Address Width Requirement

The `ADDR_WIDTH` parameter must be sufficient to address: - BASE_ADDR + (NUM_SLAVES x 64KB)

Example: For 4 slaves at BASE_ADDR = 0x8000_0000, need: - 0x8000_0000 + 0x4_0000 = 0x8004_0000 - Minimum ADDR_WIDTH: 32 bits

---

Next: Chapter 4: Interfaces

Master-Side APB Interfaces

Interface Overview

Each master port presents a complete APB slave interface, accepting transactions from an external APB master (CPU, DMA, etc.).

Signal Definition

For each master index `i` (0 to M-1):

Master Port Signal Definition

Signal	Width	Direction	Description
m<i>_apb_PSEL	1	Input	Peripheral select
m<i>_apb_PENABLE	1	Input	Transfer enable
m<i>_apb_PADDR	ADDR_WIDTH	Input	Address bus
m<i>_apb_PWRITE	1	Input	Write/read direction
m<i>_apb_PWDATA	DATA_WIDTH	Input	Write data bus
m<i>_apb_PSTRB	STRB_WIDTH	Input	Write strobes
m<i>_apb_PPROT	3	Input	Protection attributes
m<i>_apb_PRDATA	DATA_WIDTH	Output	Read data bus
m<i>_apb_PSLVERR	1	Output	Slave error response
m<i>_apb_PREADY	1	Output	Transfer ready

APB5 Sideband (mixed-version variants only)

An APB5 master port carries five additional pins. They exist ONLY on ports the generator was told are APB5 – in the shipped `apbx_xbar_2to2_mixed` that is `m1` alone, and `m0` has none of them. That structural absence IS the version gate; there is no mask to set.

APB5 Master Sideband Signals

Signal	Width	Direction	Description
m<i>_apb_PAUSER	1	Input	Request user attributes; forwarded only to an APB5 slave
m<i>_apb_PWUSER	1	Input	Write-data user

Signal	Width	Direction	Description
			attributes; likewise
m<i>_apb_PRUSER	1	Output	Completer read-data user attributes, from an APB5 slave
m<i>_apb_PBUSER	1	Output	Completer response user attributes, from an APB5 slave
m<i>_apb_PWAKEUP	1	Input	Requester wake; forwarded to every APB5 slave port – see below

Reading from an APB4 slave returns `'0` on PRUSER/PBUSER rather than a stale or fabricated value, and that is proven, not just tested (`formal/apbx_xbar/apbx_xbar_2to2_mixed/`, `ap_gate_no_fab_*`).

PWAKEUP is an input here. AMBA APB (IHI 0024E section 3.7.2, Table 3-7) defines PWAKEUP as a Requester output, and the external master is the Requester on this port. The crossbar ORs it into PWAKEUP on every APB5 slave port, without registering it, so a wake raised a cycle ahead of PSEL reaches the completer a cycle ahead of PSEL. An APB4 master has no PWAKEUP. Its transfers to an APB5 slave are still covered by the crossbar's own wake, as described in Slave Side. The forwarding is proven (`ap_pwakeup_forwarded` in `formal/apbx_xbar/apbx_xbar_2to2_mixed/`). Earlier crossbars drove this pin as an output tied to `'0`, under the old reversed direction. That was corrected in issue #102 item 7.

The user-signal widths are fixed at 1 bit, not parameterised. This table previously named AUSER_WIDTH/WUSER_WIDTH/RUSER_WIDTH/BUSER_WIDTH as though an integrator could size them. There are no such parameters: the whole module takes only ADDR_WIDTH, BASE_ADDR, DATA_WIDTH and STRB_WIDTH, and the generator hardcodes `AUSER_WIDTH(1)` .. `BUSER_WIDTH(1)` at each boundary shim. Widening a user field means regenerating the crossbar, not overriding a parameter.

Parity pins are absent entirely: every shipped variant is generated with parity off. A variant generated with `enable_parity=True` adds the IHI 0024E chapter 5 check signals on each APB5 port. Inputs are PADDRCHK, PCTRLCHK, PSELCHK, PENABLECHK, PWDATACHK, PSTRBCHK, PAUSERCHK, PWUSERCHK and PWAKEUPCHK. Outputs are PRDATACHK, PREADYCHK, PSLVERRCHK, PRUSERCHK and PBUSERCHK. All are odd parity, plus `m<i>_parity_error_wdata` and `m<i>_parity_error_ctrl`. See the GAS chapter *Mixed APB4/APB5 and Parity* for the widths.

Signal Descriptions

PSEL (Input)

Peripheral select signal. When asserted: - Indicates the master wants to access the crossbar - Must remain asserted throughout the transaction - Combined with PADDR determines target slave

PENABLE (Input)

Transfer enable signal. APB state machine: - PSEL=1, PENABLE=0: Setup phase - PSEL=1, PENABLE=1: Access phase (data transfer)

PADDR (Input)

Address bus. Used for: - Target slave selection (upper bits) - Local address within slave (lower bits) - Must be stable while PSEL asserted

PWRITE (Input)

Transfer direction: - PWRITE=1: Write transaction (data from master) - PWRITE=0: Read transaction (data to master)

PWDATA (Input)

Write data bus: - Valid during write transactions - Sampled when PREADY and PENABLE both high - Width set by DATA_WIDTH parameter

PSTRB (Input)

Write byte strobes: - One bit per byte of DATA_WIDTH - Indicates which bytes are valid in PWDATA - Ignored during read transactions

PPROT (Input)

Protection attributes (3 bits): - Bit 0: Privileged/normal access - Bit 1: Secure/non-secure access - Bit 2: Instruction/data access

PRDATA (Output)

Read data bus: - Valid during read transactions when PREADY high - Undefined during write transactions - Width matches DATA_WIDTH parameter

PSLVERR (Output)

Slave error response: - Sampled when PREADY and PENABLE both high - PSLVERR=1 indicates error condition - Propagated from the downstream slave, **or generated locally by this port** when the address decodes outside the map: on the decoding variants a miss completes here with `PSLVERR=1` and never reaches a slave (projects/components/fabric-gen-ip/lattice-apbx-xbar BUG-002, was APBX-005). The `1to1` and `2to1` variants have no decode and so never generate it locally.

PREADY (Output)

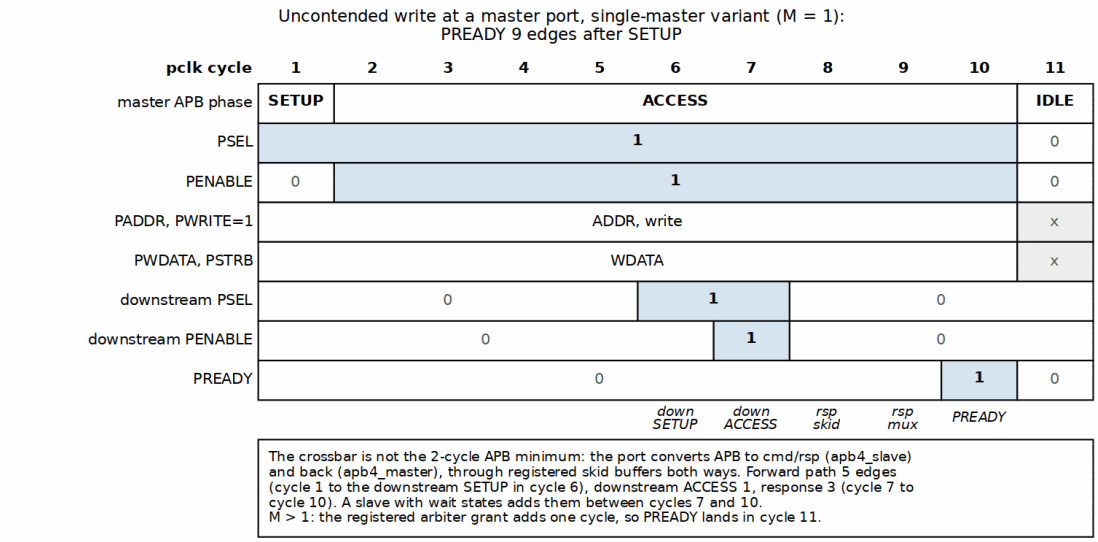
Transfer ready signal: - PREADY=1 allows the transaction to complete - PREADY=0 inserts wait states - Reflects downstream slave PREADY (or arbitration wait)

Transaction Timing

Uncontended Write (No Arbitration Wait)

Drawn to scale for a single-master variant: 9 pclk edges SETUP-to-PREADY, 10 on an arbitrated one (see 5.2).

Figure 4.1: Uncontended Write at a Master Port

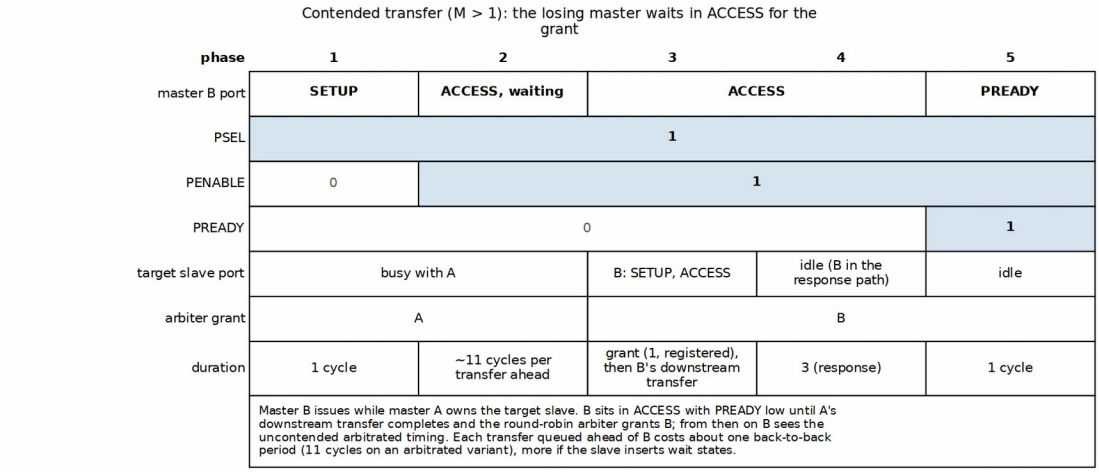


Uncontended Write at a Master Port

Contended Transaction (Arbitration Wait)

When another master holds the target slave:

Figure 4.2: Contended Transfer



Contended Transfer

Next: Slave-Side Interfaces

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

Slave-Side APB Interfaces

Interface Overview

Each slave port presents a complete APB master interface, driving transactions to an external APB slave (peripheral).

Signal Definition

For each slave index `j` (0 to N-1):

Slave Port Signal Definition

Signal	Width	Direction	Description
s<j>_apb_PSEL	1	Output	Peripheral select
s<j>_apb_PENABLE	1	Output	Transfer enable
s<j>_apb_PADDR	ADDR_WIDTH	Output	Address bus
s<j>_apb_PWRITE	1	Output	Write/read direction
s<j>_apb_PWDATA	DATA_WIDTH	Output	Write data bus
s<j>_apb_PSTRB	STRB_WIDTH	Output	Write strobes
s<j>_apb_PPROT	3	Output	Protection attributes
s<j>_apb_PRDATA	DATA_WIDTH	Input	Read data bus
s<j>_apb_PSLVERR	1	Input	Slave error response
s<j>_apb_PREADY	1	Input	Transfer ready

APB5 Sideband (mixed-version variants only)

Symmetric with the master side, and equally structural: these pins exist ONLY on slave ports the generator was told are APB5. In the shipped `apbx_xbar_2to2_mixed` that is `s0` alone – `s1` has none of them – so the absence IS the version gate, with no mask to set. Note the pairing in that variant is crosswise: the APB5 port is `m1` on the master side and `s0` here.

APB5 Slave Sideband Signals

Signal	Width	Direction	Description
s<j>_apb_PAUSER	1	Output	Request user attributes, forwarded from an APB5 master

Signal	Width	Direction	Description
s<j>_apb_PWUSER	1	Output	Write-data user attributes, likewise
s<j>_apb_PRUSER	1	Input	Read-data user attributes returned by the slave
s<j>_apb_PBUSER	1	Input	Response user attributes returned by the slave
s<j>_apb_PWAKEUP	1	Output	Crossbar wake toward the slave – see below

Widths are fixed at 1 bit. The generator hardcodes AUSER_WIDTH(1) .. BUSER_WIDTH(1) on the boundary shim and exposes no parameter for them.

s<j>_apb_PWAKEUP is driven by the crossbar. On this interface the crossbar is the Requester, and PWAKEUP is a Requester output (IHI 0024E section 3.7.2, Table 3-7). It is the OR of two terms:

- The boundary apb5_master IP's own registered wake. It is high while a command for this slave is pending or a transfer is in flight, it leads PSEL by a cycle, and it holds through PREADY. This term covers transfers from APB4 masters, which have no PWAKEUP. 3.7.2 lets a slave wait for PWAKEUP before PREADY, so a PSEL without it could deadlock.
- Every APB5 master's m<i>_apb_PWAKEUP. All masters can address all slaves, so the wake goes to every APB5 slave port, not to the decoded one. The address is not known until PSEL, and 3.7.2 recommends the wake lead PSEL.

3.7.2 names a glitch-free OR tree as a valid construction. Three properties are proven in formal/apbx_xbar/apbx_xbar_2to2_mixed/:

- ap_pwakeup_forwarded: a master's wake always reaches the slave.
- ap_pwakeup_covers_psel: the slave is never selected without PWAKEUP.
- ap_pwakeup_not_fabricated: from reset, with no master wake and no traffic, the wake stays low.

The slave's own wake is not forwarded to any master. Earlier crossbars took this pin as an input and dropped it, under the old reversed direction. That was corrected in issue #102 item 7. With parity on, the port's PWAKEUPCHK is ~s<j>_apb_PWAKEUP, computed over the OR.

Reading from an APB4 slave returns '0 on PRUSER/PBUSER rather than a stale or fabricated value, proven by ap_gate_no_fab_* in formal/apbx_xbar/apbx_xbar_2to2_mixed/.

Signal Behavior

Output Signals (Crossbar to Slave)

PSEL: Asserted when a transaction targets this slave. Remains high throughout the transaction.

PENABLE: Follows standard APB timing – exactly one setup cycle (PSEL high, PENABLE low) before ACCESS, on every transfer.

Until 2026-08-28 `apb4_master`/`apb5_master` drove **two** setup cycles when launching out of IDLE, measured `[2,2,2]` at a downstream port; only back-to-back transfers taking the ACCESS->SETUP shortcut were compliant. Fixed as AMBA TASK-071 by not asserting PSEL in the IDLE launch cycle, which cost no latency because the state sequence was unchanged. Now measured `[1,1,1,1,1]` downstream, asserted by `val/amba/test_apb_master_setup_phase.py` and by the downstream check in `dv/tests/test_apbx_xbar_timing.py`.

- Low during setup phase
- High during access phase

PADDR: Contains the full address from the master. The local address within a 64KB region is `offset[15:0]` – note this is the OFFSET's low bits, not PADDR's, so it only equals PADDR[15:0] when BASE_ADDR is 64KB-aligned (which is permitted but not required).

PWRITE, PWDATA, PSTRB, PPROT: Passed through from the winning master without modification.

Input Signals (Slave to Crossbar)

PRDATA: Captured by crossbar when both PREADY and PENABLE are high. Routed to originating master.

PSLVERR: Captured with PRDATA. Propagated to originating master as error indication.

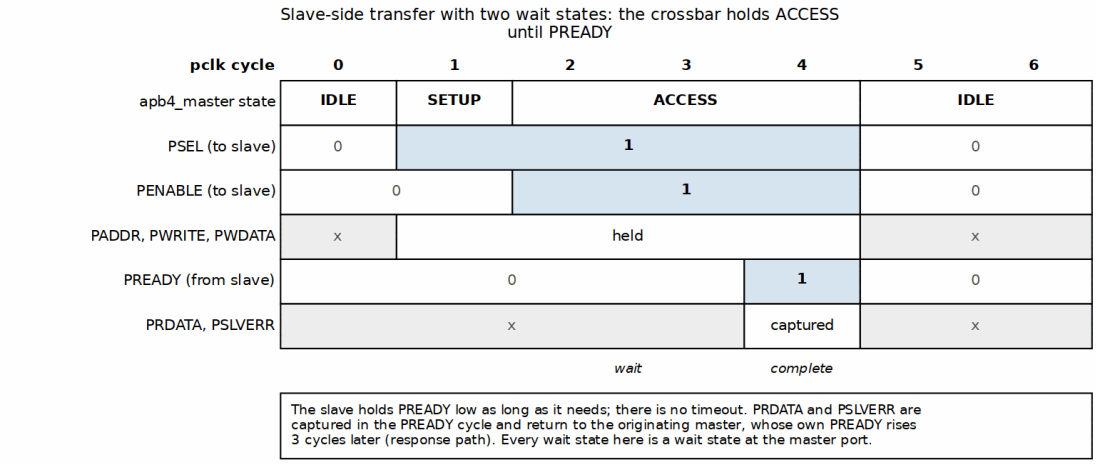
PREADY: Controls transaction completion: - PREADY=1: Transaction completes this cycle - PREADY=0: Insert wait states

Slave Response Handling

Wait States

Slaves can insert wait states by holding PREADY low:

Figure 4.3: Slave Wait States



Slave Wait States

The crossbar: - Holds master PREADY low while waiting - Does not timeout (assumes slaves always respond) - Passes through all wait states to master

Error Response

When slave asserts PSLVERR:

1. Crossbar captures PSLVERR with PRDATA
2. Routes PSLVERR to originating master
3. Master sees PSLVERR=1 when its PREADY goes high

No error handling or recovery - error indication only.

Multi-Slave Considerations

Address Overlap Prevention

Each slave port is mutually exclusive: - Only one slave receives PSEL for any given address - No overlapping address regions - Address decode is deterministic

Independent Operation

Each slave operates independently: - Own PREADY timing - No synchronization between slaves - Concurrent transactions to different slaves allowed (different masters)

Next: Clock and Reset

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

Clock and Reset

Clock Interface

Signal Definition

Clock Signal

Signal	Width	Direction	Description
`pclk`	1	Input	APB clock

Clock Requirements

Single Clock Domain: - All crossbar logic operates on the pclk rising edge - All APB signals sampled/driven synchronous to pclk - No internal clock generation or division

Frequency: - No minimum frequency requirement - Maximum frequency set by the critical path - Typical: 100-250 MHz (depends on target technology)

Duty Cycle: - No specific duty cycle requirement - Standard 50% duty cycle recommended

Reset Interface

Signal Definition

Reset Signal

Signal	Width	Direction	Description
`presetn`	1	Input	Active-low asynchronous reset

Reset Behavior

Assertion (presetn goes low): - All internal state cleared immediately - All output signals driven to safe values - Master ports: PREADY=0, PRDATA=0, PSLVERR=0 - Slave ports: PSEL=0, PENABLE=0, PADDR=0, etc.

Deassertion (presetn goes high): - Internal state begins normal operation - Reset release is NOT synchronized internally – `presetn` is applied directly as an asynchronous reset (`ALWAYS_FF_RST`) in every module. The integrator must synchronize reset DEASSERTION to `pclk` externally (recovery/removal timing) - Ready to accept transactions from the first `pclk` edge after a correctly synchronized deassertion – NOT “immediately” in the sense of the deassertion instant. Until that externally synchronized removal lands cleanly, the fabric’s state is not defined; the “Reset Deassertion” guidance later on this page (wait 1 cycle before the first transaction) is the operative rule.

Reset Safe Values

Reset Safe Values

Signal Group	Reset Value	Rationale
Master PREADY	0	apb4_slave asserts PREADY only in BUSY
Master PRDATA	0	No false data
Master PSLVERR	0	No false errors
Slave PSEL	0	No false transactions
Slave PENABLE	0	No false enables
Arbiter grants	0	No master selected

Clock-Reset Relationship

Reset Assertion

Reset can be asserted at any time: - Asynchronous assertion (immediate effect), in every build. This comes from `ALWAYS_FF_RST` in `rtl/amba/includes/reset_defs.svh`, which is unconditionally async; the old `USE_ASYNC_RESET` switch is a no-op. - **Deassertion must be synchronous to pclk** – see “Reset Deassertion” below. This bullet previously also permitted asynchronous deassertion, contradicting that section three lines later. - All in-flight transactions aborted

Reset Deassertion

For clean operation: 1. Assert reset for minimum 2 pclk cycles 2. Deassert reset synchronous to pclk rising edge 3. Wait 1 cycle before first transaction

Figure 4.4: Reset Sequence

Lattice reset: asynchronous assert, release synchronous to pclk, one idle cycle

step	1	2	3	4
phase	**reset asserted**	**released**	**wait**	**ready**
presetn	0	**1**		
duration	>= 2 pclk cycles	on a pclk rising edge	1 pclk cycle	-
internal state	reset values (cleared at once)	reset values		running
outputs	PREADY=0, PSEL=0, PENABLE=0, grants=0	safe values		per traffic
transactions	aborted / not allowed	not allowed		allowed
	Assertion is asynchronous and aborts anything in flight; the reset-safe values hold for as long as presetn is low. Release presetn synchronously to pclk, then leave one idle cycle.			

Reset Sequence

Integration Notes

Clock Domain Crossing

If masters or slaves are in different clock domains: - Insert CDC logic external to the crossbar - The crossbar assumes a single clock domain - Consider an APB clock bridge if needed

Clock Gating

The crossbar supports clock gating: - No internal activity when idle - Safe to gate pclk when no transactions pending - All registers use standard flip-flops (no clock gating cells)

Next: Chapter 5: Performance

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

Throughput Characteristics

Maximum Throughput

Single Master

With a single master accessing any slave, the figures split by arbiter presence: the generator emits an arbiter only when M > 1, and its registered grant costs a cycle on every one of them. See 5.2.

Single-Master-Active Throughput, by Variant Class

Metric	M = 1	M > 1	Notes
Cycles per transaction (uncontended)	9 SETUP->PREADY; 8 fabric	10 SETUP->PREADY; 9 fabric	measured, zero-wait slave
Sustained cycles per transaction	10 (PREADY->PREADY)	11 (PREADY->PREADY)	measured back-to-back, one active master
Maximum transactions per cycle	~0.100	~0.091	1 transaction / 10 or 11 cycles
Data throughput (32-bit @ 100MHz)	~40 MB/s	~36 MB/s	4 B per period
Data throughput (32-bit @ 250MHz)	~100 MB/s	~91 MB/s	4 B per period

M = 1: `apbx_xbar_1to1`, `apbx_xbar_1to4`. M > 1: `apbx_xbar_2to1`, `apbx_xbar_2to4`, `apbx_xbar_2to2_mixed` – so the 2x4 the PRD calls the typical SoC case is the slower column. This table published the M = 1 figures unconditionally until 2026-08-29.

Multi-Master (Uncontended)

When masters access different slaves: - Each master achieves single-master throughput - Aggregate throughput = M x single-master throughput - No CONTENTION overhead – but the registered grant still costs one cycle on every M > 1 variant, contended or not (see 5.2). Only the single-master variants have no arbiter at all.

Multi-Master (Contended)

When masters compete for the same slave:

Contended Access Throughput

Masters	Effective Throughput	Notes
2	50% per master	Round-robin sharing
3	33% per master	Round-robin sharing
4	25% per master	Round-robin sharing

Arbitration Handoff

Grant Persistence

The crossbar implements grant persistence: - A master holds the grant for the duration of ONE transaction - The grant is released at the RESPONSE handshake, not when PSEL drops (`grant_ack = grant && rsp_valid && rsp_ready`, arbiter instantiated with `WAIT_GNT_ACK(1)`), after which the round-robin mask rotates – holding PSEL does not keep the grant if another master is requesting - A single master with no competitor re-wins immediately, so its transactions still stream back-to-back – at the fabric's own cadence, not at APB's 2-cycle minimum

Back-to-Back Timing

A "zero-bubble" 4-cycle two-transaction diagram appeared here in earlier revisions. It described a pipeline this RTL does not have: the `apb4_slave` front end is a one-command-at-a-time FSM (IDLE -> BUSY -> WAIT), so the next command is not captured until the previous transaction completes. But sustained cadence is not equal to single-transfer latency either – it is one cycle longer. A master holding PSEL high and dropping PENABLE for exactly one cycle after each PREADY (the earliest LEGAL turnaround) measures **PREADY-to-PREADY** = **10 pclk cycles**, against a 9-cycle SETUP-to-PREADY single transfer. The extra cycle is the next transfer's mandatory SETUP phase, which cannot overlap the previous transfer's ACCESS. See 5.2 for the convention and the cycle-by-cycle breakdown. There is still no overlap between consecutive transactions to draw.

Throughput Factors

Factors That Reduce Throughput

Throughput Reduction Factors

Factor	Impact	Mitigation
Slave wait states	Variable	Choose faster peripherals
Arbitration conflicts	~11 cycles per master ahead in the round-robin	Distribute access across slaves
Slave response time	Variable	Design slaves for quick response

Optimal Usage Pattern

For maximum throughput: 1. Distribute master access across different slaves 2. Minimize slave wait states 3. Use burst-like access patterns (same master, same slave) 4. Avoid rapid master switching on same slave

---

Next: Latency Analysis

Latency Analysis

Transaction Latency

Uncontended Access

When a master has exclusive access to a slave:

Uncontended Transaction Latency

Phase	Cycles	Description
Master SETUP	1	PSEL high, PENABLE low
apb4_slave capture + cmd skid	2	command registered out
apb4_master IDLE-launch / SETUP / ACCESS	3	downstream PSEL then PENABLE
Response + rsp skid	2	slave PREADY through the return skid
apb4_slave BUSY -> PREADY	1	master-visible completion
Total	**9**	PSEL to PREADY (8 of them ACCESS to PREADY)

These are measured numbers, not protocol minimums. A crossbar transfer is NOT the 2-cycle APB minimum: the fabric converts APB to an internal cmd/rsp protocol through `apb4_slave` and back through `apb4_master`, and both directions cross REGISTERED skid buffers.

Measurement convention. Every figure below counts rising `pclk` edges and names the two edges it spans. State the edges or the number means nothing – this page has been wrong twice from not doing so.

The numbers depend on whether the variant has an arbiter, and the generator only emits one when there is more than one master. A single-master variant has no `arbiter_round_robin` at all; a multi-master variant has one per slave, and its grant is a flop (`grant <= w_next_grant`), which costs exactly one cycle on every quantity below.

Measured Timing, Always-Ready Slave

Quantity	M = 1	M > 1	Spans
Fabric latency	**8**	**9**	first ACCESS edge

Quantity	M = 1	M > 1	Spans
			-> edge where PREADY is high
Single-transfer latency	**9**	**10**	SETUP edge -> edge where PREADY is high
Back-to-back period	**10**	**11**	PREADY edge -> next PREADY edge

M = 1 is `apbx_xbar_1to1` and `apbx_xbar_1to4` (0 arbiters). M > 1 is `apbx_xbar_2to1`, `apbx_xbar_2to4` and `apbx_xbar_2to2_mixed` (1, 4 and 2 arbiters). All six columns are measured, not derived – see `dv/tests/test_apbx_xbar_timing.py`, which asserts both classes.

Until 2026-08-29 this page published the M = 1 numbers unconditionally, under the heading "Uncontended Access", having measured only `apbx_xbar_1to1`. They were one cycle optimistic for every arbitrated variant – including the 2x4 that the PRD calls the typical SoC case.

The period is 10, not 9, and the difference is structural. After PREADY at cycle N the bus is still in ACCESS for that cycle, so the next transfer's mandatory SETUP cycle cannot begin before N+1; its ACCESS follows at N+2 and its PREADY at N+2+8 = N+10. Reaching 9 would require a SETUP cycle overlapping the previous transfer's ACCESS, which is not a legal APB waveform. So sustained cadence does **not** equal single-transfer latency here – it is exactly one cycle longer.

Earlier revisions of this page claimed 2 cycles (the bare APB minimum for a directly-attached slave, which does not apply through this fabric), and later claimed a 9-cycle sustained cadence, which came from a probe whose turnaround was not legal APB. A qc reviewer flagged the 9-cycle cadence and was told it was a false positive; the reviewer was right. `dv/tests/test_apbx_xbar_timing.py` now asserts all three numbers against the RTL so the question is settled by the suite rather than by argument.

Contended Access

When multiple masters compete for the same slave:

Arbitration Latency

Scenario	Additional Latency	Description
Win arbitration	1 cycle	grant is REGISTERED in arbiter_round_robin
Lose to 1 master	~10+ cycles	wait one full transaction (see above)
Lose to N masters	~10N+ cycles	wait N transactions

Latency Breakdown

Forward Path (Master to Slave)

Forward Path Latency

Component	M = 1	M > 1	Type
apb4_slave capture	1	1	Registered
apb4_slave cmd skid out	1	1	Registered
Address decode	0	0	Combinational
Arbitration	**0**	**1**	No arbiter is emitted when M = 1; when one is, its grant is a flop
apb4_master cmd skid in	1	1	Registered
apb4_master cmd skid out	1	1	Registered
apb4_master IDLE -> SETUP	1	1	FSM
Total	**5**	**6**	master SETUP edge to downstream PSEL edge

Both columns measured: 5 on `apbx_xbar_1to1`, 6 on `apbx_xbar_2to1`. The arbitration row is the whole difference between the two variant classes. This table carried a single unconditional column with a 1-cycle arbitration row and a 5-cycle total until 2026-08-29 – which cannot both be true, since the 5 was measured on the variant that has no arbiter.

The three numbers on this page must sum, and now do:

M = 1 M > 1 5 forward 6 forward + 1 downstream SETUP -> ACCESS + 1 downstream SETUP -> ACCESS + 3 response + 3 response = 9 SETUP-to-PREADY = 10 SETUP-to-PREADY

(With a zero-wait slave the downstream ACCESS and PREADY edges are the same cycle, which is why the response row can be read from either. A slave with wait states adds its own cycles between them and shifts the total.)

This table read **4** until 2026-08-29, which could not be reconciled with the page's own 9-cycle total – 4 + 3 left two cycles unaccounted for. Measured on `apbx_xbar_1to1` with an always-ready slave: master SETUP at cycle 1, downstream PSEL at cycle 6, downstream ACCESS at cycle 7, master PREADY at cycle 10.

Response Path (Slave to Master)

Response Path Latency

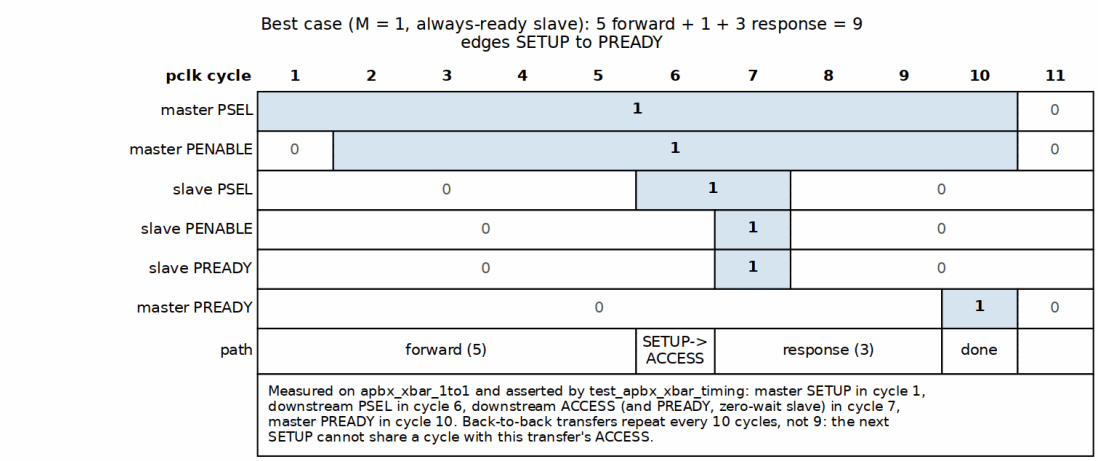
Component	Cycles	Type
Slave response	Variable	The slave's own PREADY timing, excluded below
apb4_master captures into rsp skid	1	Registered
rsp skid out, through the response mux	1	Skid registered; the mux itself is combinational
apb4_slave drives PREADY	1	Registered
Total	**3**	downstream PREADY edge to master PREADY edge

Same on both variant classes – arbitration is on the command path only. The rows read "0-1 cycle" each until 2026-08-29, which could not sum to the stated 3.

Latency Timing Diagram

Best Case (No Waits, No Contention)

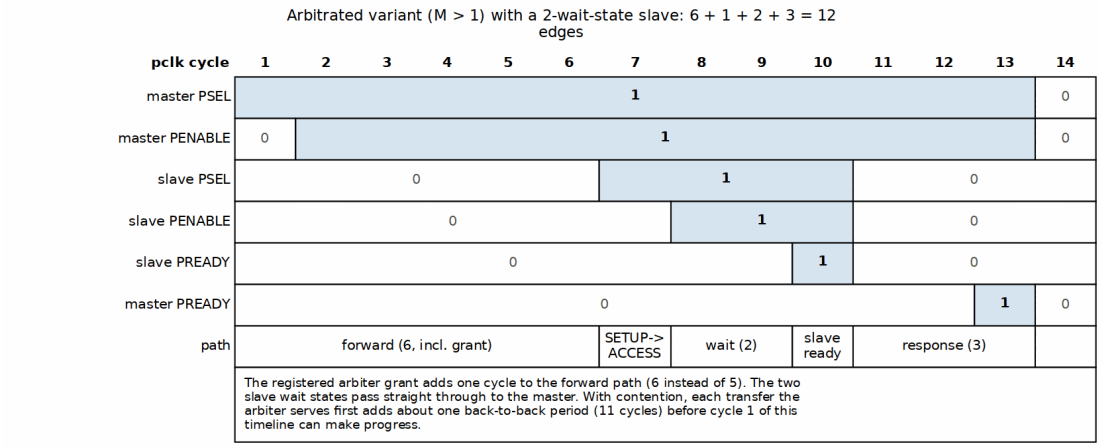
Figure 5.1: Best-Case Latency Timeline



Best-Case Latency Timeline

Worst Case (Contention + Slave Wait States)

Figure 5.2: Arbitration and Slave Wait States



Arbitration and Slave Wait States

Latency Optimization

Design Recommendations

Latency Optimization Recommendations

Goal	Recommendation
Minimize contention	Use more slaves, spread access
Reduce arbitration wait	Lower master count per slave
Reduce slave latency	Design slaves with quick PREADY

Next: Resource Estimates

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

Resource Estimates

FPGA Resource Usage

Pre-Generated Variants

Estimated resource usage for common configurations (32-bit data width):

Variant	LUTs	FFs	Description
apbx_xbar_1to1	~150	~80	Full 1x1 with buffering
apbx_xbar_2to1	~300	~150	2 masters, arbitration
apbx_xbar_1to4	~400	~200	4 slaves, decode
apbx_xbar_2to4	~800	~400	Full 2x4 crossbar

Hand estimates, not synthesis results, and the single source of truth for this component. PRD §9.3 previously carried a second, different set of numbers (2-3x smaller in every cell); that table now refers here rather than competing with it. Nothing in the repo has been synthesized to confirm either set – treat these as order-of-magnitude only.

: FPGA Resource Estimates (32-bit)

Scaling Factors

Resource usage scales approximately as:

Resource Scaling Factors

Component	Scaling	Notes
apb4_slave instances	M x base	Linear with masters
apb4_master instances	N x base	Linear with slaves
Arbiters	N x log2(M)	Per slave, sized for masters
Address decode	N	Comparators per slave
Response mux	N x M	Full crosspoint for

Component	Scaling	Notes
		responses

Lines of Code

Source code metrics for pre-generated variants:

Source Code Metrics

Variant	Lines of Code	Complexity
apbx_xbar_1to1	165	Low
apbx_xbar_2to1	314	Medium
apbx_xbar_1to4	417	Medium
apbx_xbar_2to2_mixed	625	Medium
apbx_xbar_2to4	806	Medium-High

This table is the single source of truth for LOC, as it already is for the LUT/FF estimates above. Exact `wc -l` of `rtl/*.sv`, measured 2026-10-12; the PRD and the key-features page point here rather than carrying their own copies.

Three tables disagreed before that – by up to 33% on the same file (2to4 as ~1000, ~751 and 751) – because each was corrected on a different day. Even the most precise-looking set was stale: 165/314/384/751/570 was right when reconciled on 2026-08-27 and wrong two days later, once the projects/components/fabric-gen-ip/lattice-apbx-xbar BUG-002 (was APBX-005) decode-miss logic added lines. Numbers that must be hand-synced across three pages will desync; keeping one and pointing at it is the fix.

Power Considerations

Static Power

Static power scales with: - Total register count (FFs) - Routing resources used - Technology node

Dynamic Power

Dynamic power depends on: - Transaction rate (activity factor) - Data width (toggling bits) - Clock frequency

Power Optimization

Power Optimization Techniques

Technique	Benefit	Trade-off
Clock gating	Reduce idle power	Adds gating logic
Lower data width	Fewer toggling bits	Reduced throughput
Fewer slaves	Less decode logic	Reduced flexibility

Area Comparison

Relative area comparison (normalized to apbx_xbar_1to1):

Relative Area Comparison

Variant	Relative Area
apbx_xbar_1to1	1.0x (baseline)
apbx_xbar_2to1	2.0x
apbx_xbar_1to4	2.5x
apbx_xbar_2to4	5.0x

Custom Configuration Estimation

For custom MxN configuration:

```
Estimated LUTs ~= 50 + (M * 75) + (N * 50) + (M * N * 10)
Estimated FFs  ~= 20 + (M * 40) + (N * 30)
```

Example: 4x8 crossbar - LUTs: 50 + (4*75)* + (*8*50) + (4*8*10) = 50 + 300 + 400 + 320 = ~1070 - FFs: 20 + (*4*40)* + (*8*30) = 20 + 160 + 240 = ~420

Next: Chapter 6: Integration

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

System Requirements

Hardware Requirements

Clock and Reset

Clock and Reset Requirements

Requirement	Specification
Clock input	Single clock (pclk)
Clock frequency	No minimum, technology-dependent maximum
Reset input	Active-low (`presetn`); asynchronous assertion, synchronous deassertion
Reset duration	Minimum 2 clock cycles

APB Compliance

The crossbar requires APB-compliant masters and slaves:

APB Compliance Requirements

Requirement	Description
Protocol version	APB4 (with PSTRB, PPROT); APB5 sideband supported only on mixed-version variants such as `apbx_xbar_2to2_mixed`
Wait states	Supported (PREADY-based)
Error response	Supported (PSLVERR)
Slave timeout	Not handled internally

Software Requirements

Address Map Configuration

Software must be configured with the correct address map:

```
// Example address definitions
#define PERIPHERAL_BASE     0x10000000
#define UART_BASE           (PERIPHERAL_BASE + 0x00000)
#define GPIO_BASE           (PERIPHERAL_BASE + 0x10000)
#define TIMER_BASE          (PERIPHERAL_BASE + 0x20000)
#define SPI_BASE            (PERIPHERAL_BASE + 0x30000)
```

Driver Considerations

Driver Considerations

Consideration	Recommendation
Polling vs interrupt	Both supported
Atomic operations	Not guaranteed across arbitration
Cache coherency	APB typically uncached

Constraints

Address Map Constraints

Address Map Constraints

Constraint	Value	Notes
Slave region size	64KB in the shipped variants	Set at generation via `--slave-size`; fixed once generated
Maximum slaves	16	Generator limit
Maximum masters	16	Generator limit
Address alignment	Byte aligned	No alignment requirement

Transaction Constraints

Transaction Constraints

Constraint	Value	Notes
Outstanding transactions	1 per master	APB protocol limit
Transaction interleaving	Not supported	APB is non-pipelined
Burst transactions	Not supported	APB single-beat only

Integration Checklist

Pre-Integration

- ☐ Verify BASE_ADDR does not conflict with other subsystems
- ☐ Confirm all slaves fit within 64KB regions
- ☐ Validate ADDR_WIDTH is sufficient for address map
- ☐ Check DATA_WIDTH matches system data width

Post-Integration

- ☐ Verify address decode routing to correct slaves
- ☐ Test all master-to-slave paths
- ☐ Validate arbitration behavior (if multi-master)
- ☐ Check reset behavior

Next: Parameter Configuration

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

Parameter Configuration

Module Parameters

Core Parameters

Core Parameter Definition

Parameter	Type	Default	Range	Description
ADDR_WIDTH	int	32	18-64 for decoding variants; 1-64 for 1to1/2to1	Address bus width in bits. The decode part-select reads bits `16 +: $clog2(S)`, so a 2-slave variant needs only bit 16 – but the generator emits the range-check span as a fixed 32-bit constant (`paddr < BASE_ADDR + 32'h...`), so anything below 18 width-expands and fails a warnings-fatal build. 17 lints clean only with `-Wno-fatal`; it is not usable.
DATA_WIDTH	int	32	8, 16, 32, 64	Data bus width in bits
STRB_WIDTH	int	DATA_WIDTH/8	derived	Write strobe width. Declared as a parameter, but it is computed from DATA_WIDTH and overriding it is not supported
BASE_ADDR	logic[AD	0x10000000	any	Base of the slave

Parameter	Type	Default	Range	Description
	DR_WIDTH-1:0]		address; the slave index is decoded from the OFFSET, so no span alignment is required	map. **UNUSED on `1to1` and `2to1`** – they have a single slave and no decode, so the parameter is declared only for interface uniformity and setting it changes nothing (see those modules' RTL headers)

Derived Parameters

These are calculated automatically:

Derived Parameters

Parameter	Derivation	Description
STRB_WIDTH	DATA_WIDTH / 8	Byte strobe width
(none)	—	Each variant declares only the four parameters above; master/slave counts are baked into the module name and port list, NOT exposed as parameters.

Parameter Usage Examples

Default Configuration

```
apbx_xbar_2to4 u_xbar (
    .pclk(apb_clk),
    .presetn(apb_rst_n),
    // ... port connections
);
// Uses defaults: ADDR_WIDTH=32, DATA_WIDTH=32, BASE_ADDR=0x10000000
```

Custom Base Address

```
apbx_xbar_2to4 #(
    .BASE_ADDR(32'h4000_0000)
) u_xbar (
    .pclk(apb_clk),
    .presetn(apb_rst_n),
    // ... port connections
);
// Slaves at 0x4000_0000, 0x4001_0000, 0x4002_0000, 0x4003_0000
```

64-bit Data Width

```
apbx_xbar_2to4 #(
    .DATA_WIDTH(64)
) u_xbar (
    .pclk(apb_clk),
    .presetn(apb_rst_n),
    // ... port connections with 64-bit data buses
);
// STRB_WIDTH automatically becomes 8
```

Wide Address

```
apbx_xbar_2to4 #(
    .ADDR_WIDTH(40),
    .BASE_ADDR(40'h80_0000_0000)
) u_xbar (
    .pclk(apb_clk),
    .presetn(apb_rst_n),
    // ... port connections with 40-bit address buses
);
```

Parameter Validation

Address Width Check

Ensure ADDR_WIDTH can accommodate the full address range:

```
Required minimum: log2(BASE_ADDR + NUM_SLAVES * 64KB)
```

Example: 4 slaves at BASE_ADDR=0x1000_0000 - Max address: 0x1000_0000 + 4*0x10000 = 0x1004_0000 - Required: 29 bits minimum - ADDR_WIDTH=32 is sufficient

Data Width Constraints

Valid Data Width Configurations

DATA_WIDTH	STRB_WIDTH	Valid APB Widths
8	1	Yes
16	2	Yes
32	4	Yes (most common)
64	8	Yes

Custom Generation Parameters

When using the Python generator for custom configurations:

```
python generate_xbars.py \
    --masters 3 \
    --slaves 6 \
    --base-addr 0x80000000 \
    --slave-size 0x10000
```

Generator Command Line Options

Option	Description	Default
–masters	Number of master ports	Required
–slaves	Number of slave ports	Required
–base-addr	Base address (hex)	0x10000000
–slave-size	Address space per slave (0x1000 = 4KB, 0x10000 = 64KB)	0x10000

`--slave-size` decides which address bits select the slave: the crossbar decodes `offset[log2(slave_size) +: ceil(log2(S))]`, so 64KB windows give `offset[16 +: n]` and 4KB windows give `offset[12 +: n]`. Until 2026-09-07 this script did not pass the value through on the custom path, so a custom variant silently came out with 4KB windows while all four shipped variants had 64KB – one family, two address maps. It now defaults to 64KB to match them.

The lower-level `apbx_xbar_generator.py` takes the same flag but keeps its own historical default of 4KB; prefer `generate_xbars.py` so the family stays consistent.

Next: Verification Strategy

Verification Strategy

Pre-Verified Component

The Lattice is pre-verified:

Test / Variant	Transactions	Status	Coverage
apbx_xbar_1to1	100+	Pass	Basic connectivity
apbx_xbar_2to1	130+	Pass	Arbitration stress
apbx_xbar_1to4	200+	Pass	Address decode
apbx_xbar_2to4	350+	Pass	Full crossbar stress
apbx_xbar_2to2_mixed	8 directed	Pass	All four APB4/APB5 pairings, sideband gating, decode miss, structural pin check
apbx_xbar_timing	2 variants	Pass	Published latency/cadence figures, both variant classes

Overall: 100% passing (9/9), >750 transactions plus the directed mixed-version and timing checks. `apbx_xbar_2to2_mixed` also has a formal harness (`formal/apbx_xbar/apbx_xbar_2to2_mixed/`).

: Pre-Verification Status

Test Categories

Basic Connectivity

Tests for each variant include: - Single read transaction to each slave - Single write transaction to each slave - Read-modify-write sequences - All address regions exercised

Address Decode Verification

For multi-slave variants: - Verify each slave selected by correct address range - Boundary address testing - Invalid address behavior (decoding variants complete the access with PSLVERR; they do not hang – see 4.1)

Arbitration Testing

For multi-master variants: - Round-robin fairness verification - Contention handling - Grant persistence validation - No starvation testing

Stress Testing

High-intensity scenarios: - Back-to-back transactions - Random delays - Mixed read/write patterns - Concurrent multi-master access

APB5 Sideband Gating (mixed variant)

For `apbx_xbar_2to2_mixed`: - All four APB4/APB5 master/slave pairings exercised - Sideband signals flow only where both ends are APB5 - APB4 ports carry no sideband pins (structural check) - Decode-miss completes with `PSLVERR`

Timing and Protocol Compliance

`test_apbx_xbar_timing` asserts the published figures against the RTL: - Single-transfer latency for single-master and arbitrated classes - Back-to-back cadence (PREADY-to-PREADY) - Forward-path and response-path breakdown - Downstream setup-phase count (`[1,1,1,1,1]`)

Integration Testing Recommendations

Level 1: Basic Integration

Level 1 Integration Tests

Test	Description	Pass Criteria
Reset test	Apply reset, verify outputs	All outputs at safe values
Single transaction	Read from each slave	Correct data returned
Write verify	Write and readback	Data matches

Level 2: Functional Integration

Level 2 Integration Tests

Test	Description	Pass Criteria
All masters	Each master accesses each slave	No errors
Arbitration	Concurrent master access	Fair grant rotation
Error propagation	Force PSLVERR from slave	Error reaches master

Level 3: System Integration

Level 3 Integration Tests

Test	Description	Pass Criteria
CPU access	Software reads/writes peripherals	Correct behavior
DMA access	DMA transfers via crossbar	Data integrity
Mixed traffic	CPU and DMA concurrent	No deadlock, fair access

Verification Collateral

Available Test Infrastructure

Test Infrastructure

Item	Location	Description
CocoTB tests	`dv/tests/`	Python testbenches
Wave configs	`dv/tests/GTKW/`	GTKWave configurations
Coverage	`dv/tests/coverage_reports/`	Merged functional coverage reports
Build orchestration	`dv/tests/Makefile`	Area Makefile delegating to `make/tests.mk`
Formal harness (mixed variant)	`formal/apbx_xbar/apbx_xbar_2to2_mixed/`	Sideband-gating proofs

Running Verification

Tests are run through the area `Makefile` (which delegates to the shared `make/tests.mk` infrastructure); bare `pytest` is not the supported entry point because the Makefile supplies filelists, levels, and coverage options.

```
cd projects/components/fabric-gen-ip/lattice-apbx-xbar/dv/tests/

# Run all crossbar tests -- clean-all first for a run you intend to
trust
make clean-all && make run-all-gate

# Run specific variant
make run-apbx_xbar_2to4-gate

# Run with waveforms (WAVES=1 via the target)
make run-apbx_xbar_2to4-gate-waves
# create_view_cmd() writes the gtkwave command beside the log
```

Known Limitations

Not Tested

Known Limitations

Scenario	Reason	Recommendation
Slave timeout	Not implemented	Add external watchdog
>16 masters/slaves	Generator limit	Custom modification needed
Non-64KB regions	Not supported	Use multiple crossbars

End of Hardware Architecture Specification

Related Documentation

- **Lattice MAS** - Detailed implementation
- **PRD.md** - Product requirements
- **README.md** - Quick start guide

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

Lattice Micro-Architecture Specification Index

Component: Lattice (MxN Interconnect) **Version:** 1.2 **Date:** 2026-10-12 **Purpose:** Detailed micro-architecture specification for Lattice component

---

Document Organization

This specification covers the Lattice: a parametric MxN interconnect that routes multiple APB masters to multiple APB slaves with automatic address-based routing and round-robin arbitration.

Main Documentation

README.md

Chapter 1: Architecture

chapters/01_architecture.md

Chapter 2: Address Decode and Arbitration

chapters/02_address_and_arbitration.md

Chapter 3: RTL Generator

chapters/03_rtl_generator.md

Quick Navigation

For New Users

1. Start with README.md for overview and quick start
2. Read 01_architecture.md to understand the design
3. Study 02_address_and_arbitration.md for operational details
4. Reference 03_rtl_generator.md if you need a custom configuration

For Integration

- **Pre-generated variants:** See 01_architecture.md Section “Pre-Generated Variants”
- **Custom generation:** See 03_rtl_generator.md Section “Quick Start”
- **Address mapping:** See 02_address_and_arbitration.md Section “Address Decode”
- **Arbitration behavior:** See 02_address_and_arbitration.md Section “Arbitration”

Common Questions

All answered in README.md, section “Common Questions”.

Visual Assets

Every diagram in this specification is checked in as both source and rendered output:

- **Source Files:**
  - `assets/graphviz/*.dot` - Graphviz source diagrams
  - `assets/wavedrom/*.json` - WaveJSON timing diagrams
- **Rendered Files:**
  - `assets/graphviz/*.png` - PNG format (for document generation)

Architecture Diagrams

1. **Lattice Architecture (2x4 Example)**
   - Source: assets/graphviz/apbx_xbar_architecture.dot
   - Rendered: assets/graphviz/apbx_xbar_architecture.png
2. **Address Decode Flow**
   - Source: assets/graphviz/address_decode_flow.dot
   - Rendered: assets/graphviz/address_decode_flow.png

Timing Diagrams

1. **Round-Robin Arbitration**
   - Source: assets/wavedrom/arbitration_round_robin.json
   - Rendered: assets/wavedrom/arbitration_round_robin.png

Component Overview

Key Features

- **Parametric MxN Configuration:** Any combination of M masters and N slaves (up to 16x16)
- **Automatic Address Decode:** 64KB per slave, simple offset-based routing
- **Round-Robin Arbitration:** Per-slave fair arbitration, no master starvation
- **Back-to-Back Transactions:** Accepted without master-side idle cycles, though they do not overlap inside the fabric (10 pclk cycles each for M = 1, 11 for arbitrated variants; one more than that variant's single-transfer latency — see HAS 5.2)
- **Grant Persistence:** Grant held through transaction completion
- **RTL Generation:** Python-based generator for custom configurations

Pre-Generated Variants

Module	M×N	Use Case
apbx_xbar_1to1	1×1	Passthrough, protocol conversion
apbx_xbar_2to1	2×1	Multi-master arbitration
apbx_xbar_1to4	1×4	Simple SoC peripheral bus
apbx_xbar_2to4	2×4	Typical SoC with CPU+DMA
apbx_xbar_2to2_mixed	2×2	Mixed APB4/APB5 (m0=APB4, m1=APB5; s0=APB5, s1=APB4)

Design Philosophy

Proven Building Blocks: - Built from production-tested `apb4_slave` and `apb4_master` modules - No new protocol logic - pure composition - Each component independently verified

Parametric Generation: - Generator creates any MxN configuration - Pre-generated common variants for fast integration - Custom variants generated on-demand

Clean Separation: - Master-side: APB slaves convert protocol → cmd/rsp - Internal: Arbitration + address decoding - Slave-side: APB masters convert cmd/rsp → protocol

---

Related Documentation

Companion Specifications

- **Lattice HAS** - Hardware Architecture Specification (high-level)

Project-Level

- **PRD.md:** ../../PRD.md - Complete product requirements document
- **CLAUDE.md:** ../../CLAUDE.md - AI assistant integration guide
- **README.md:** ../../README.md - Quick start guide

Test Infrastructure

- **Test Directory:** `../../dv/tests/` - CocoTB + pytest test suite
- **Test Results:** 100% passing (9/9) — all pre-generated variants plus the dedicated timing test; `apbx_xbar_2to2_mixed` additionally has a formal harness

RTL

- **Core Modules:** `../../rtl/apbx_xbar_*.sv` - Pre-generated crossbars
- **Wrappers:** `../../rtl/wrappers/` - testbench scaffolds (pclk/presetn only), not integration wrappers
- **Generator:** `../../bin/generate_xbars.py` - Python generator script

Version History

Version 1.0 (2025-10-25): - Initial specification release - Complete visual documentation (3 diagrams) - Generator documentation - All pre-generated variants verified (100% passing)

Version 1.1 (2026-10-12): - Refreshed against current RTL, generator, and DV - Updated verification status (9/9 tests passing, mixed-variant formal harness) - Added `--slave-size` CLI option to generator table - Updated line-of-code counts (single source of truth remains HAS ch05_performance/03_resources.md)

Last Updated: 2026-10-12 **Maintained By:** RTL Design Sherpa Project

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

Product Requirements Document (PRD)

Lattice Generator

Version: 1.0 **Date:** 2025-10-19 **Status:** Production Ready **Owner:** RTL Design Sherpa Project
Parent Document: `/PRD.md` **Codename:** lattice — directory `lattice-apbx-xbar` (fabric-gen-ip family with `bridge` and `delta`); RTL modules keep the `apbx_xbar_*` prefix

---

1. Executive Summary

The **Lattice** is a parametric generator for configurable MxN APB interconnect fabrics connecting multiple APB masters to multiple APB slaves. It is built from the proven `apb4_slave` and `apb4_master` modules: independent round-robin arbitration per slave, automatic address-based routing, and no new protocol logic.

1.1 Quick Stats

- **Modules:** 5 pre-generated variants + generator for custom sizes
- **Max Capacity:** Up to 16x16 (configurable)
- **Architecture:** apb4_slave → arbitration + decode → apb4_master
- **Status:** Production ready, all tests passing
- **Generator:** Python-based parametric code generation

1.2 Project Goals

- **Primary:** Provide production-quality APB interconnect for SoC integration
- **Secondary:** Demonstrate parametric RTL generation best practices
- **Tertiary:** Support both fixed and dynamically-generated variants

---

2. Key Design Principles

2.1 Architecture Philosophy

Proven Building Blocks: - Uses `apb4_slave.sv` and `apb4_master.sv` from rtl/amba/apb4 - Each module independently tested and production-proven - Crossbar = composition of proven components

Parametric Generation: - Generator creates any MxN configuration - Pre-generated common variants (1to1, 2to1, 1to4, 2to4, 2to2_mixed) - Custom variants generated on-demand

Clean Separation: - Master-side: APB slaves convert protocol to cmd/rsp - Internal: Arbitration + address decoding - Slave-side: APB masters convert cmd/rsp back to protocol

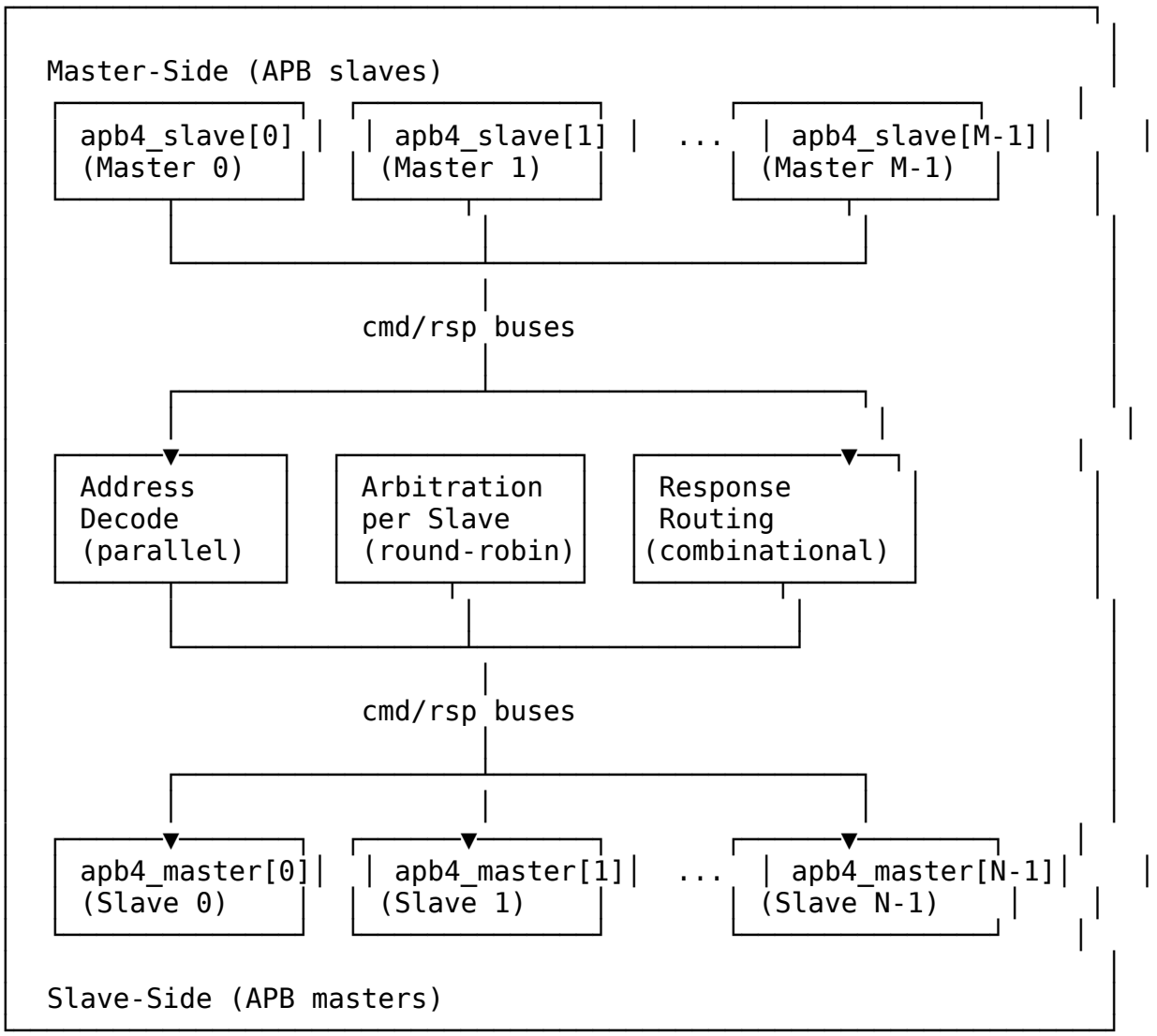
2.2 Design Trade-offs

Feature	Choice	Rationale
Arbitration	Round-robin per slave	Fair, simple, predictable
Address Map	64KB per slave	Sufficient for most peripherals
Grant Persistence	Hold through response	One transaction per grant; mask rotates after
Address Decode	Parallel decode	Low latency, simple logic

3. Architecture Overview

3.1 Top-Level Block Diagram

Lattice (M masters × N slaves)



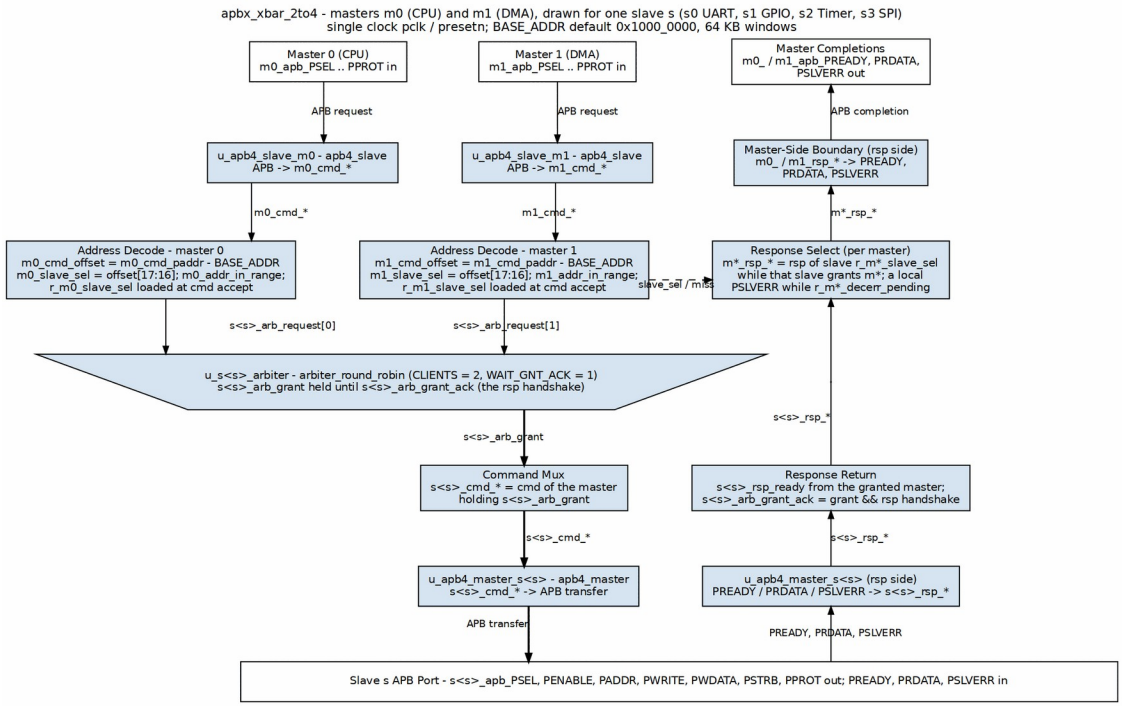


Figure: Lattice top-level architecture showing 2 masters connected to 4 slaves. Source: docs/lattice_mas/assets/graphviz/apbx_xbar_architecture.dot

3.2 Address Mapping

```
BASE_ADDR + 0x0000_0000 → 0x0000_FFFF : Slave 0 (64KB)
BASE_ADDR + 0x0001_0000 → 0x0001_FFFF : Slave 1 (64KB)
BASE_ADDR + 0x0002_0000 → 0x0002_FFFF : Slave 2 (64KB)
BASE_ADDR + 0x0003_0000 → 0x0003_FFFF : Slave 3 (64KB)
...
BASE_ADDR + 0x000F_0000 → 0x000F_FFFF : Slave 15 (64KB)
```

Default BASE_ADDR: 0x1000_0000

Address Decode Example:

The crossbar extracts the slave index from the upper bits of the address offset:

```
slave_index = (address - BASE_ADDR) >> 16  // Divide by 64KB (0x10000)
```

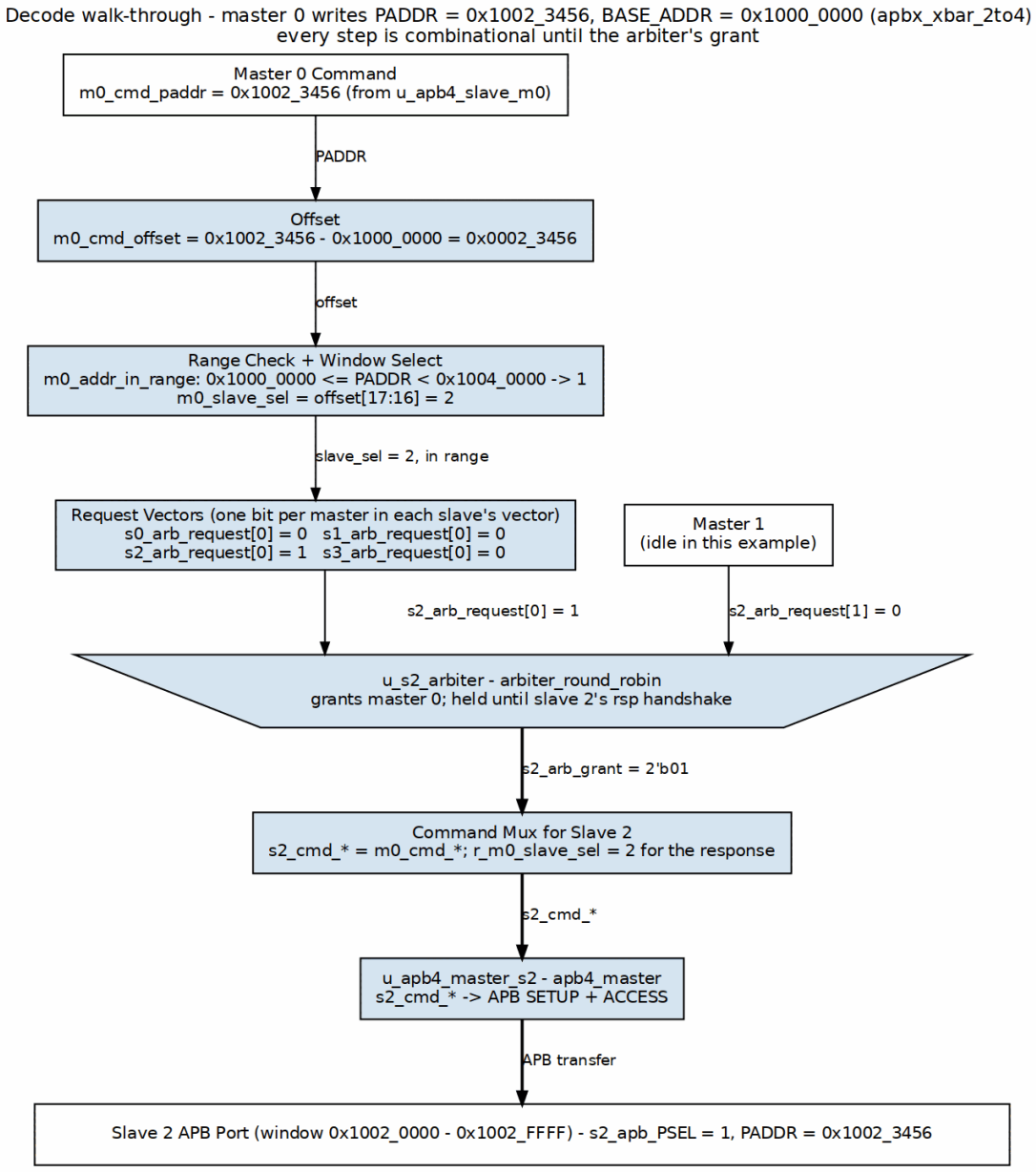


Figure: Address decode flow showing how address 0x10023456 routes to Slave 2. Source: docs/lattice_mas/assets/graphviz/address_decode_flow.dot

3.3 Arbitration Strategy

Per-Slave Round-Robin: - Each slave has independent arbiter - Master priority rotates after each grant - Grant held from command acceptance through response completion - No master can starve another master

Example:

```
Slave 0 accessed by M0, M1, M0 → Next grant goes to M1
Slave 1 accessed by M1, M1, M0 → Next grant goes to M1
  (the winner cannot immediately re-win: after an M0 grant the
   mask leaves only M1 eligible)
```

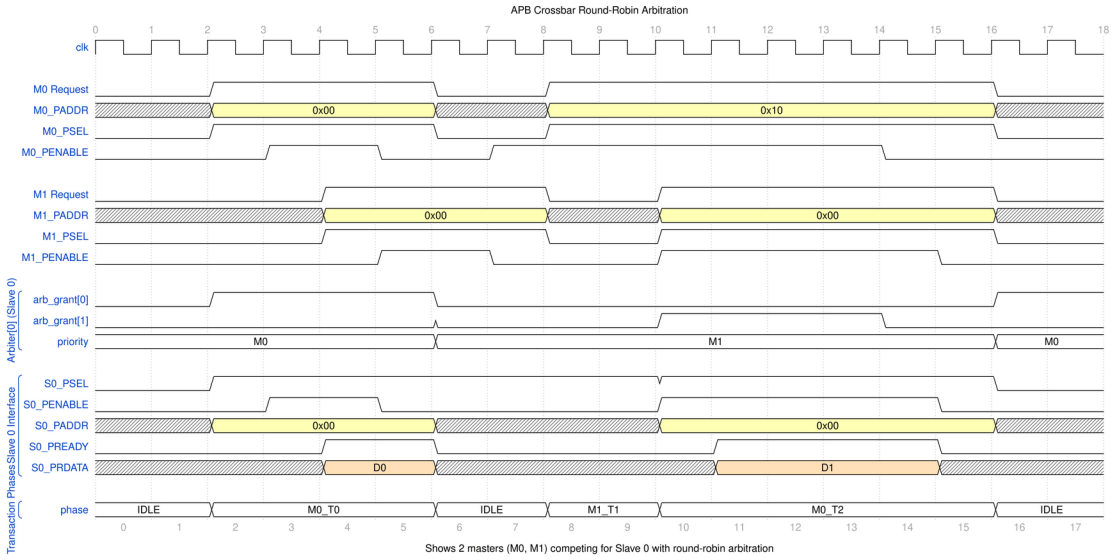


Figure: Round-robin arbitration timing showing 2 masters competing for Slave 0. Master priority rotates after each grant to ensure fair access. *Source: docs/lattice_mas/assets/wavedrom/arbitration_round_robin.json*

---

4. Available Variants

4.1 Pre-Generated Modules

Module	Masters	Slaves	Use Case
apbx_xbar_1to1	1	1	Protocol conversion, testing
apbx_xbar_2to1	2	1	Multi-master arbitration
apbx_xbar_1to4	1	4	Address decode, simple SoC
apbx_xbar_2to4	2	4	Full crossbar, typical SoC
apbx_xbar_2to2_mixed	2	2	Mixed APB4/APB5 (s0 is APB5)

LOC figures live in HAS ch05_performance/03_resources.md, which is the single source of truth for them. A duplicate column here disagreed with that table by up to 33%.

4.2 Wrapper Modules

These are TESTBENCH SCAFFOLDS, not integration building blocks. Each exposes `pclk` and `presetn` and nothing else — every APB signal is an internal net driven by the cocotb testbench. They exist because the four legacy tests drive a wrapper rather than the bare crossbar; a design cannot connect anything to one.

Wrapper	Configuration	Purpose
`apbx_xbar_1to1_wrap`	1×1	Test scaffold for `test_apbx_xbar_1to1.py`
`apbx_xbar_2to1_wrap`	2×1	Test scaffold for `test_apbx_xbar_2to1.py`
`apbx_xbar_1to4_wrap`	1×4	Test scaffold for `test_apbx_xbar_1to4.py`
`apbx_xbar_2to4_wrap`	2×4	Test scaffold for `test_apbx_xbar_2t`

Wrapper	Configuration	Purpose
		`o4.py`

To integrate, instantiate the crossbar itself. `apbx_xbar_2to2_mixed` has no wrapper precisely because its test drives the DUT directly, which is the better pattern.

---

5. Functional Requirements

FR-1: Arbitrary MxN Configuration

Priority: P0 (Critical) **Status:** Implemented and verified

Description: Generate crossbar for any M masters and N slaves (up to 16x16)

Verification: Generator creates valid RTL for all tested configurations

FR-2: Round-Robin Arbitration

Priority: P0 (Critical) **Status:** Implemented and verified

Description: Fair per-slave arbitration with rotating master priority

Verification: Arbitration stress tests (130+ transactions for 2to1)

FR-3: Address-Based Routing

Priority: P0 (Critical) **Status:** Implemented and verified

Description: Automatic slave selection based on address ranges

Verification: Address decode validation (200+ transactions for 1to4)

FR-4: Back-to-Back Transactions

Priority: P1 (High) **Status:** Implemented and verified

Description: Consecutive transactions are accepted with no master-side idle cycles required. They do NOT overlap inside the fabric — `apb4_slave` is one-command-at-a-time. Sustained cadence is 10 pclk cycles PREADY-to-PREADY for a single-master variant and 11 for an arbitrated one, in each case one more than that variant's SETUP-to-PREADY latency of 9 or 10 (HAS 5.1/5.2).

Verification: Performance tests show consecutive transactions

FR-5: Configurable Address Map

Priority: P1 (High) **Status:** Implemented and verified

Description: BASE_ADDR parameter sets base of address map

Verification: Tests with multiple BASE_ADDR values

---

6. Interface Specifications

6.1 Master-Side Interface (APB Slave)

Per-master APB slave interface:

```
input  logic                      m<i>_apb_PSEL
input  logic                      m<i>_apb_PENABLE
input  logic [ADDR_WIDTH-1:0]     m<i>_apb_PADDR
input  logic                      m<i>_apb_PWRITE
input  logic [DATA_WIDTH-1:0]     m<i>_apb_PWDATA
input  logic [STRB_WIDTH-1:0]     m<i>_apb_PSTRB
input  logic [2:0]                m<i>_apb_PPROT
output logic [DATA_WIDTH-1:0]     m<i>_apb_PRDATA
output logic                      m<i>_apb_PSLVERR
output logic                      m<i>_apb_PREADY
```

6.2 Slave-Side Interface (APB Master)

Per-slave APB master interface:

```
output logic                      s<j>_apb_PSEL
output logic                      s<j>_apb_PENABLE
output logic [ADDR_WIDTH-1:0]     s<j>_apb_PADDR
output logic                      s<j>_apb_PWRITE
output logic [DATA_WIDTH-1:0]     s<j>_apb_PWDATA
output logic [STRB_WIDTH-1:0]     s<j>_apb_PSTRB
output logic [2:0]                s<j>_apb_PPROT
input  logic [DATA_WIDTH-1:0]     s<j>_apb_PRDATA
input  logic                      s<j>_apb_PSLVERR
input  logic                      s<j>_apb_PREADY
```

---

7. Parameter Configuration

Parameter	Type	Default	Range	Description
ADDR_WIDTH	int	32	18-64 for multi-slave variants (the decode part-select needs bit 17); 1-64 for 1to1/2to1	Address bus width
DATA_WIDTH	int	32	8,16,32,64	Data bus width
STRB_WIDTH	int	DATA_WIDTH/8	-	Strobe width (auto-calc)
BASE_ADDR	logic [ADDR_WIDTH-1:0]	0x10000000	Any except the top S x 64KB, where BASE_ADDR + span wraps and the range check can never pass	Base address for slave map. Inert on 1to1/2to1 (no decode)

8. Generator Usage

8.1 Pre-Generated Variants

Use existing modules directly:

```
# Available in
projects/components/fabric-gen-ip/lattice-apbx-xbar/rtl/
apbx_xbar_1to1.sv
apbx_xbar_2to1.sv
apbx_xbar_1to4.sv
apbx_xbar_2to4.sv
apbx_xbar_2to2_mixed.sv
```

8.2 Custom Generation

Generate custom MxN crossbar:

```
cd projects/components/fabric-gen-ip/lattice-apbx-xbar/bin/
python generate_xbars.py --masters 3 --slaves 6
python generate_xbars.py --masters 4 --slaves 8 --base-addr 0x80000000
```

8.3 Generator Options

```
python generate_xbars.py --help

Options:
  --masters M       Number of masters (1-16)
  --slaves N        Number of slaves (1-16)
  --base-addr ADDR  Base address (default: 0x10000000)
```

---

9. Performance Characteristics

9.1 Latency

Path	M = 1	M > 1	Notes
Command path	6	7	master SETUP (1) + forward path (5, or 6 with a registered arbiter grant)
Response path	3	3	arbitration is on the command side only
Total	**9**	**10**	SETUP->PREADY, uncontended, zero-wait slave (8 / 9 ACCESS->PREADY)

M = 1 is `1to1` and `1to4`, which the generator emits with no arbiter at all. M > 1 is `2to1`, `2to4` and `2to2_mixed` — including the 2x4 this document calls the typical SoC case. This table gave only the M = 1 figures, unlabelled, until 2026-08-30.

APB's 2-cycle minimum applies to a directly-attached slave, not through this fabric: the crossbar converts APB→cmd/rsp→APB across registered skid buffers in both directions. See HAS 5.2 for the cycle-by-cycle breakdown and the measurement.

9.2 Throughput

- **Back-to-back transactions:** Supported, but not overlapped — `apb4_slave` is a one-command-at-a-time FSM, so the next command is captured only after the previous transaction completes
- **Maximum rate:** ~1 transaction per 10 pclk cycles (1to1, 1to4) or 11 (2to1, 2to4, 2to2_mixed) per master (measured back-to-back at an always-ready slave)

9.3 Resource Utilization (Estimated)

See HAS ch05_performance/03_resources.md — that table is the single source of truth. An earlier duplicate here quoted figures 2-3x smaller in every cell; both were unsourced hand estimates, so the duplicate is withdrawn rather than reconciled.

10. Verification Status

10.1 Test Coverage

Test	Transactions	Status	Coverage
test_apbx_xbar_1to1	100+	Pass	Basic connectivity
test_apbx_xbar_2to1	130+	Pass	Arbitration stress
test_apbx_xbar_1to4	200+	Pass	Address decode
test_apbx_xbar_2to4	350+	Pass	Full crossbar stress
test_apbx_xbar_2to2_mixed	8 directed	Pass	All four APB4/APB5 pairings, sideband gating, decode miss, structural pin check
test_apbx_xbar_timing	2 variants	Pass	Published latency/cadence figures, both variant classes

Overall: 100% passing (9/9), >750 transactions plus the directed mixed-version and timing checks. `2to2_mixed` also has a formal harness (`formal/apbx_xbar/apbx_xbar_2to2_mixed/`).

10.2 Test Methodology

Test Structure: - CocoTB framework - Random transaction generation - Variable delay profiles - Address range validation - Arbitration fairness checks

Verification Points: - Correct address routing - Round-robin arbitration - Response integrity - Back-to-back transactions - Error propagation

---

11. Integration Guide

11.1 Simple Integration (1to4)

```
apbx_xbar_1to4 #(
    .ADDR_WIDTH(32),
    .DATA_WIDTH(32),
    .BASE_ADDR(32'h1000_0000)
) u_xbar (
    .pclk(apb_clk),
    .presetn(apb_rst_n),

    // Master interface
    .m0_apb_PSEL(cpu_psel),
    .m0_apb_PENABLE(cpu_penable),
    .m0_apb_PADDR(cpu_paddr),
    .m0_apb_PWRITE(cpu_pwrite),
    .m0_apb_PWDATA(cpu_pwdata),
    .m0_apb_PSTRB(cpu_pstrb),
    .m0_apb_PPROT(cpu_pprot),
    .m0_apb_PRDATA(cpu_prdata),
    .m0_apb_PSLVERR(cpu_pslverr),
    .m0_apb_PREADY(cpu_pready),

    // Slave 0: UART (0x1000_0000 - 0x1000_FFFF)
    .s0_apb_PSEL(uart_psel),
    .s0_apb_PENABLE(uart_penable),
    // ... (similar connections)

    // Slave 1: GPIO (0x1001_0000 - 0x1001_FFFF)
    .s1_apb_PSEL(gpio_psel),
    // ... (similar connections)

    // Slave 2: Timer (0x1002_0000 - 0x1002_FFFF)
    // Slave 3: SPI (0x1003_0000 - 0x1003_FFFF)
);
```

11.2 Multi-Master Integration (2to4)

```
apbx_xbar_2to4 #(
    .ADDR_WIDTH(32),
    .DATA_WIDTH(32),
    .BASE_ADDR(32'h1000_0000)
) u_xbar (
    .pclk(apb_clk),
    .presetn(apb_rst_n),

    // Master 0: CPU
```

```
    .m0_apb_PSEL(cpu_psel),
    // ... (full interface)

    // Master 1: DMA
    .m1_apb_PSEL(dma_psel),
    // ... (full interface)

    // Slaves 0-3: Peripherals
    // ... (slave connections)
);
```

---

12. Design Constraints

12.1 Limitations

Constraint	Value	Rationale
Max masters	16	Generator limit (configurable)
Max slaves	16	Generator limit (configurable)
Slave region size	64KB in every shipped variant	Baked in at generation (`--slave-size`, default 4KB); not overridable at elaboration
No slave disable	N/A	All slaves always active
No timeout	N/A	Assumes slaves always respond

12.2 Assumptions

1. **Slave responses:** All slaves respond eventually (no timeout handling)
2. **Address ranges:** Slaves occupy 64KB regions starting at BASE_ADDR
3. **APB compliance:** All masters and slaves follow APB protocol
4. **Single clock domain:** All signals synchronous to pclk

13. Future Enhancements

13.1 Potential Features

- **Configurable slave region sizes:** Not just 64KB
- **Slave enable/disable:** Dynamic slave activation
- **Timeout detection:** Watchdog for hung slaves
- **Transaction ordering:** Optional in-order completion
- **Priority arbitration:** Weighted instead of round-robin

13.2 Generator Improvements

- **HDL output formats:** Support Verilog, VHDL, Chisel
- **Automated testing:** Generate tests alongside RTL
- **Documentation generation:** Auto-generate integration guides
- **Synthesis scripts:** Generate vendor-specific scripts

---

14. References

14.1 Internal Documentation

- `README.md` - Quick start guide
- `CLAUDE.md` - AI assistant guidelines
- `bin/generate_xbars.py` - Generator script
- `dv/tests/` - Test implementations

14.2 External Standards

- **AMBA APB Protocol v2.0** - ARM IHI 0024C
- **APB4 Extensions** - AMBA 5 specification

14.3 Related Components

- `rtl/amba/apb4/apb4_slave.sv` - Master-side building block
- `rtl/amba/apb4/apb4_master.sv` - Slave-side building block
- `projects/components/fabric-gen-ip/lattice-apbx-xbar/bin/apbx_xbar_generator.py` - Main generator

Document Version: 1.0 **Last Review:** 2025-10-19 **Next Review:** 2026-01-01 **Maintained By:** RTL Design Sherpa Project

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

Claude Code Guide: Lattice Component

Version: 1.0 **Last Updated:** 2025-10-19 **Purpose:** AI-specific guidance for working with Lattice component

Codename: lattice · **Directory:** `lattice-apbx-xbar` (fabric-gen-ip family with `bridge` and `delta`); RTL modules keep the `apbx_xbar_*` prefix

---

Quick Context

What: Lattice Generator - Parametric APB interconnect for connecting M masters to N slaves
Status: Production Ready (all tests passing) **Your Role:** Help users generate, integrate, and customize APB crossbars

Complete Specification:
`projects/components/fabric-gen-ip/lattice-apbx-xbar/PRD.md` ← **Always reference this for technical details**

Critical Rules for This Component

Rule #0: This is a GENERATOR, Not Just Modules

IMPORTANT: Lattice is both: 1. **Pre-generated modules** (1to1, 2to1, 1to4, 2to4, 2to2_mixed) in `rtl/` 2. **Python generator** (`bin/generate_xbars.py`) for custom configurations

When users ask for crossbar: - Check if pre-generated variant exists first - Only suggest generation if custom size needed

Decision Tree:

```
User needs crossbar?
├─ 1x1, 2x1, 1x4, 2x4? → Use pre-generated module
├─ Mixed APB4/APB5 2x2? → Use apbx_xbar_2to2_mixed
└─ Custom MxN? → Run generator script
```

Rule #1: Address Map is Fixed Per-Slave

Each slave occupies 64KB region:

```
Slave 0: BASE_ADDR + 0x0000_0000 → 0x0000_FFFF
Slave 1: BASE_ADDR + 0x0001_0000 → 0x0001_FFFF
Slave 2: BASE_ADDR + 0x0002_0000 → 0x0002_FFFF
...
```

The window size is not a module parameter, but it IS a generator option. Every *shipped* variant uses 64KB windows because `bin/generate_xbars.py` passes `slave_size=0x10000`. The generator's own default is 4KB, and `apbx_xbar_generator.py --slave-size` accepts any power of 2 >= 256 bytes. Once generated the size is baked into the decode part-select, so it cannot be overridden at elaboration — only `BASE_ADDR` can, and only on the decoding variants.

If user asks for different sizes:

```
WRONG: "Let me modify the generator to support custom sizes per slave"

CORRECT: "The shipped variants use 64KB per slave. You can:
1. Use BASE_ADDR to shift the whole map (decoding variants only)
2. For a different window, regenerate with --slave-size (power of 2,
>= 256B)
3. Or use multiple crossbars with different BASE_ADDR values"
```

Rule #2: Know the Pre-Generated Variants

Available in `rtl/` directory:

Module	M×N	Use Case	When to Recommend
apbx_xbar_1to1	1×1	Passthrough	Protocol conversion, testing
apbx_xbar_2to1	2×1	Arbitration	Multi-master to single peripheral
apbx_xbar_1to4	1×4	Decode	Single CPU to multiple peripherals
apbx_xbar_2to4	2×4	Full crossbar	CPU + DMA to peripherals
apbx_xbar_2to2_mixed	2×2	Mixed APB4/APB5	APB5 peripheral alongside APB4

Always suggest pre-generated first!

Rule #3: Generator Syntax

Generate custom crossbar:

```
cd projects/components/fabric-gen-ip/lattice-apbx-xbar/bin/
python generate_xbars.py --masters 3 --slaves 6
```

Options:

```
--masters M         Number of masters (1-16)
--slaves N          Number of slaves (1-16)
--base-addr ADDR    Base address (default: 0x10000000)
```

Example:

```
# Generate 3x8 crossbar with custom base
python generate_xbars.py --masters 3 --slaves 8 --base-addr 0x80000000

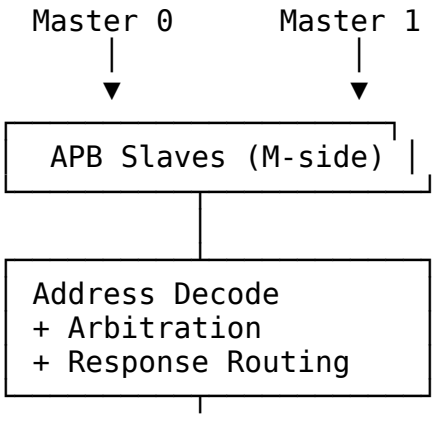
# Generate a 5x5 variant
python generate_xbars.py --masters 5 --slaves 5
```

Architecture Quick Reference

Module Organization

```
projects/components/fabric-gen-ip/lattice-apbx-xbar/
├── rtl/
│   ├── apbx_xbar_1to1.sv        Core crossbar modules
│   ├── apbx_xbar_2to1.sv
│   ├── apbx_xbar_1to4.sv
│   ├── apbx_xbar_2to4.sv
│   ├── apbx_xbar_2to2_mixed.sv  (m0=APB4, m1=APB5, s0=APB5, s1=APB4)
│   └── wrappers/               testbench scaffolds, not integration
wrappers
│       ├── apbx_xbar_1to1_wrap.sv
│       ├── apbx_xbar_2to1_wrap.sv
│       ├── apbx_xbar_1to4_wrap.sv
│       └── apbx_xbar_2to4_wrap.sv
├── bin/
│   └── generate_xbars.py       Generator script
├── dv/
│   └── tests/
│       ├── test_apbx_xbar_1to1.py
│       ├── test_apbx_xbar_2to1.py
│       ├── test_apbx_xbar_1to4.py
│       ├── test_apbx_xbar_2to4.py
│       ├── test_apbx_xbar_2to2_mixed.py
│       ├── test_apbx_xbar_timing.py     published latency/cadence
figures
│       ├── test_apbx_xbar_parity.py     generates a parity-on all-
APB5 2x2 (APB5 *CHK, PWAKEUP)
│       └── GTKW/                Waveform configs
├── PRD.md                       Complete specification
├── CLAUDE.md                    This file
└── README.md                    Quick start guide
```

Block Diagram (2×4 Example)




APB Masters (S-side)
(more)
Slave 0
Slave 1 ... Slave 3

Common User Questions and Responses

Q: “How do I connect a CPU to 4 peripherals?”

A: Use the pre-generated 1to4 crossbar:

```
apbx_xbar_1to4 #(
    .ADDR_WIDTH(32),
    .DATA_WIDTH(32),
    .BASE_ADDR(32'h1000_0000)  // Start of peripheral space
) u_peripheral_xbar (
    .pclk(apb_clk),
    .presetn(apb_rst_n),

    // CPU connection (master 0)
    .m0_apb_PSEL(cpu_psel),
    .m0_apb_PENABLE(cpu_penable),
    .m0_apb_PADDR(cpu_paddr),
    .m0_apb_PWRITE(cpu_pwrite),
    .m0_apb_PWDATA(cpu_pwdata),
    .m0_apb_PSTRB(cpu_pstrb),
    .m0_apb_PPROT(cpu_pprot),
    .m0_apb_PRDATA(cpu_prdata),
    .m0_apb_PSLVERR(cpu_pslverr),
    .m0_apb_PREADY(cpu_pready),

    // Peripheral 0: UART @ 0x1000_0000
    .s0_apb_PSEL(uart_psel),
    .s0_apb_PENABLE(uart_penable),
    // ... (full interface)

    // Peripheral 1: GPIO @ 0x1001_0000
    // Peripheral 2: Timer @ 0x1002_0000
    // Peripheral 3: SPI @ 0x1003_0000
);
```

Address map: - UART: 0x1000_0000 - 0x1000_FFFF - GPIO: 0x1001_0000 - 0x1001_FFFF - Timer: 0x1002_0000 - 0x1002_FFFF - SPI: 0x1003_0000 - 0x1003_FFFF

See: PRD.md Section 11.1

Q: “I need CPU and DMA to access peripherals. Which crossbar?”

A: Use the 2to4 crossbar:

```
apbx_xbar_2to4 #(
    .ADDR_WIDTH(32),
    .DATA_WIDTH(32),
```

```
    .BASE_ADDR(32'h1000_0000)
) u_soc_xbar (
    .pclk(apb_clk),
    .presetn(apb_rst_n),

    // Master 0: CPU
    .m0_apb_PSEL(cpu_psel),
    // ... (full interface)

    // Master 1: DMA
    .m1_apb_PSEL(dma_psel),
    // ... (full interface)

    // Slaves 0-3: Peripherals
    // ... (slave connections)
);
```

Key feature: Round-robin arbitration per slave ensures fair access between CPU and DMA.

See: PRD.md Section 11.2

Q: "What if I need 3 masters and 8 slaves?"

A: Generate custom crossbar:

```
cd projects/components/fabric-gen-ip/lattice-apbx-xbar/bin/
python generate_xbars.py --masters 3 --slaves 8   # writes
apbx_xbar_3to8.sv next to the script

# Check generated file, then move it into rtl/
ls -lh apbx_xbar_3to8.sv
mv apbx_xbar_3to8.sv ../rtl/
```

Then instantiate like any other crossbar:

```
apbx_xbar_3to8 #(
    .ADDR_WIDTH(32),
    .DATA_WIDTH(32),
    .BASE_ADDR(32'h1000_0000)
) u_custom_xbar (
    // ... connections for 3 masters and 8 slaves
);
```

See: PRD.md Section 8

Q: "How does arbitration work?"

A: Per-slave round-robin:

Each slave has its own arbiter that rotates master priority:

```
Example with 2 masters accessing Slave 0:

Transaction 1: M0 requests → M0 granted
Transaction 2: M0 and M1 request → M1 granted (rotated)
Transaction 3: M1 requests → M1 granted
Transaction 4: M0 and M1 request → M0 granted (rotated)
```

Key points: - **Independent per slave:** Each slave arbitrates independently - **Fair:** No master can starve another - **Grant persistence:** Once granted, master holds slave until response completes - **Back-to-back:** supported, but not overlapped — 10 pclk cycles per transfer for 1to1/1to4, 11 for the arbitrated variants (2to1, 2to4, 2to2_mixed), PREADY-to-PREADY (see HAS 5.1/5.2)

See: PRD.md Section 3.3

Q: "Can I change the address map?"

A: BASE_ADDR parameter shifts entire map, but per-slave size is fixed:

```
// Default: 0x1000_0000
apbx_xbar_1to4 #(.BASE_ADDR(32'h1000_0000)) u_xbar1 (...);
// Slaves at: 0x1000_0000, 0x1001_0000, 0x1002_0000, 0x1003_0000

// Shifted: 0x8000_0000
apbx_xbar_1to4 #(.BASE_ADDR(32'h8000_0000)) u_xbar2 (...);
// Slaves at: 0x8000_0000, 0x8001_0000, 0x8002_0000, 0x8003_0000
```

Limitation: Each slave always occupies 64KB (0x10000 bytes)

Workaround for different sizes: 1. Use multiple crossbars with different BASE_ADDR 2. Modify generator's `addr_offset` calculation 3. Use address masking in slaves

See: PRD.md Section 3.2

Q: "How do I run tests?"

A: Use the area Makefile (never bare pytest):

```
cd projects/components/fabric-gen-ip/lattice-apbx-xbar/dv/tests/

# Run specific test
make run-apbx_xbar_1to4-gate

# Run all tests -- clean-all first for a run you intend to trust
make clean-all && make run-all-gate

# Run with waveforms (WAVES=1 via the target, not --vcd)
```

```
make run-apbx_xbar_2to4-gate-waves
# create_view_cmd() writes the gtkwave command beside the log
```

Test coverage: - **test_apbx_xbar_1to1**: 100+ transactions (passthrough) - **test_apbx_xbar_2to1**: 130+ transactions (arbitration) - **test_apbx_xbar_1to4**: 200+ transactions (address decode) - **test_apbx_xbar_2to4**: 350+ transactions (full stress) - **test_apbx_xbar_2to2_mixed**: 8 directed — all four APB4/APB5 pairings, sideband gating, decode miss, structural pin check; plus `_pwakeup`: PWAKEUP forwarded requester -> completer (IHI 0024E 3.7.2) - **test_apbx_xbar_parity**: no shipped variant enables parity, so this generates one into its sim_build and checks odd `*CHK` parity (IHI 0024E ch. 5), error injection, and PWAKEUP forwarding - **test_apbx_xbar_timing**: both variant classes — asserts the published latency, cadence and per-path breakdown so the numbers cannot drift

All tests passing (22/22 at func, 2026-10-12) — and `2to2_mixed` additionally has a formal harness at `formal/apbx_xbar/apbx_xbar_2to2_mixed/`.

See: `PRD.md` Section 10

Integration Patterns

Pattern 1: Simple Peripheral Interconnect

```
// CPU to 4 peripherals
apbx_xbar_1to4 #(
    .ADDR_WIDTH(32),
    .DATA_WIDTH(32),
    .BASE_ADDR(32'h1000_0000)
) u_periph_xbar (
    .pclk(sys_clk),
    .presetn(sys_rst_n),

    // CPU side
    .m0_apb_PSEL(cpu_apb_psel),
    .m0_apb_PENABLE(cpu_apb_penable),
    .m0_apb_PADDR(cpu_apb_paddr),
    .m0_apb_PWRITE(cpu_apb_pwrite),
    .m0_apb_PWDATA(cpu_apb_pwdata),
    .m0_apb_PSTRB(cpu_apb_pstrb),
    .m0_apb_PPROT(cpu_apb_pprot),
    .m0_apb_PRDATA(cpu_apb_prdata),
    .m0_apb_PSLVERR(cpu_apb_pslverr),
    .m0_apb_PREADY(cpu_apb_pready),

    // Peripherals
    .s0_apb_PSEL(uart_psel), /* ... full interface ... */
    .s1_apb_PSEL(gpio_psel), /* ... */
    .s2_apb_PSEL(timer_psel), /* ... */
    .s3_apb_PSEL(spi_psel) /* ... */
);
```

Pattern 2: Multi-Master System

```
// CPU + DMA to peripherals
apbx_xbar_2to4 #(
    .ADDR_WIDTH(32),
    .DATA_WIDTH(32),
    .BASE_ADDR(32'h4000_0000)
) u_soc_xbar (
    .pclk(apb_clk),
    .presetn(apb_rst_n),

    // Master 0: CPU
    .m0_apb_PSEL(cpu_psel),
    /* ... full CPU interface ... */

    // Master 1: DMA Controller
```

```
    .m1_apb_PSEL(dma_psel),
    /* ... full DMA interface ... */

    // Slave 0-3: Memory-mapped peripherals
    .s0_apb_PSEL(mem_ctrl_psel), /* ... */
    .s1_apb_PSEL(uart_psel), /* ... */
    .s2_apb_PSEL(i2c_psel), /* ... */
    .s3_apb_PSEL(adc_psel) /* ... */
);
```

Pattern 3: Hierarchical Interconnect

```
// Top-level crossbar
apbx_xbar_1to4 u_top_xbar (
    // .m0_apb_PSEL(cpu_apb_PSEL), .m0_apb_PENABLE(...), ...  (elided)
    .s0_apb_PSEL(periph_bus0_PSEL), /* ...rest of the port... */  //
To sub-crossbar 0
    .s1_apb_PSEL(periph_bus1_PSEL), /* ...rest of the port... */  //
To sub-crossbar 1
    .s2_apb_PSEL(mem_ctrl_PSEL), /* ...rest of the port... */     //
Direct to memory controller
    .s3_apb_PSEL(dma_ctrl_PSEL), /* ...rest of the port... */      //
Direct to DMA
);

// Sub-crossbar 0 for low-speed peripherals
apbx_xbar_1to4 u_periph_xbar0 (
    .m0_apb_PSEL(periph_bus0_PSEL), /* ...rest of the port... */
    .s0_apb_PSEL(uart0_PSEL), /* ...rest of the port... */
    .s1_apb_PSEL(gpio0_PSEL), /* ...rest of the port... */
    .s2_apb_PSEL(i2c0_PSEL), /* ...rest of the port... */
    .s3_apb_PSEL(spi0_PSEL), /* ...rest of the port... */
);

// Sub-crossbar 1 for high-speed peripherals
apbx_xbar_1to4 u_periph_xbar1 (
    .m0_apb_PSEL(periph_bus1_PSEL), /* ...rest of the port... */
    .s0_apb_PSEL(uart1_PSEL), /* ...rest of the port... */
    .s1_apb_PSEL(timer_PSEL), /* ...rest of the port... */
    .s2_apb_PSEL(pwm_PSEL), /* ...rest of the port... */
    .s3_apb_PSEL(adc_PSEL), /* ...rest of the port... */
);
```

Anti-Patterns to Catch

Anti-Pattern 1: Generating When Pre-Generated Exists

```
WRONG:
User: "I need a 2×4 crossbar"
You: "Let me generate that for you..."
python generate_xbars.py --masters 2 --slaves 4

CORRECTED:
"Use the pre-generated apbx_xbar_2to4.sv in the rtl/ directory.
No generation needed!"
```

Anti-Pattern 2: Assuming Custom Per-Slave Sizes

```
WRONG:
User: "Can I make slave 0 256KB and slave 1 4KB?"
You: "Sure, let me modify the parameters..."

CORRECTED:
"The shipped variants use 64KB per slave. For a different window:
1. Regenerate with --slave-size (power of 2, >= 256B)
2. Or use multiple crossbars with different BASE_ADDR
3. Or implement address masking in slaves"
```

Anti-Pattern 3: Not Mentioning Address Map

```
WRONG:
User: "How do I integrate the crossbar?"
You: *Shows port connections only*

CORRECTED:
"Here's the integration with address map:
apbx_xbar_1to4 #(.BASE_ADDR(32'h1000_0000)) u_xbar (...);

Address map:
- Slave 0: 0x1000_0000 - 0x1000_FFFF
- Slave 1: 0x1001_0000 - 0x1001_FFFF
- Slave 2: 0x1002_0000 - 0x1002_FFFF
- Slave 3: 0x1003_0000 - 0x1003_FFFF"
```

Anti-Pattern 4: Forgetting About Wrappers

```
WRONG:
User: "I need a quick 10×10 crossbar"
You: "Run the generator with --masters 10 --slaves 10"

CORRECTED:
"We have pre-configured wrappers in rtl/wrappers/:
```

```
- apbx_xbar_1to1_wrap.sv, apbx_xbar_2to1_wrap.sv,
  apbx_xbar_1to4_wrap.sv, apbx_xbar_2to4_wrap.sv

Those four are TESTBENCH SCAFFOLDS, not integration wrappers: each has
only
pclk and presetn, with every APB signal internal to the cocotb bench,
so a
design cannot connect to one. Instantiate the crossbar directly. For
any
other MxS, generate one with bin/generate_xbars.py -- and there is no
m10_s10 wrapper in the tree."
```

Debugging Workflow

Issue: Address Not Routing Correctly

Check in order: 1. BASE_ADDR parameter set correctly? 2. Address within slave's 64KB region? 3. PSEL signal asserted by master? 4. All APB signals properly connected?

Calculate expected slave:

```
slave_index = (address - BASE_ADDR) >> 16  // Divide by 64KB
```

Debug commands:

```
cd dv/tests && make run-apbx_xbar_1to4-gate-waves
# gtkwave command is written beside the log by create_view_cmd()
```

Issue: Arbitration Not Fair

Symptoms: - One master dominates - Other masters starved

Check: 1. Arbiters instantiated per slave? 2. Round-robin logic correct? 3. Grant persistence working?

Verify with tests:

```
cd dv/tests && make run-apbx_xbar_2to1-gate   # Arbitration stress
test
```

Issue: Back-to-Back Transactions Stalling

Check: 1. Grant persistence enabled? 2. Slaves responding with PREADY? 3. No unintended pipeline bubbles?

View waveforms:

```
cd dv/tests && make run-apbx_xbar_2to4-gate-waves
# gtkwave command is written beside the log by create_view_cmd()
```

Quick Commands

```
# List available crossbars
ls projects/components/fabric-gen-ip/lattice-apbx-xbar/rtl/*.sv

# Generate custom crossbar
cd projects/components/fabric-gen-ip/lattice-apbx-xbar/bin/
python generate_xbars.py --masters 3 --slaves 6

# Run all tests
cd projects/components/fabric-gen-ip/lattice-apbx-xbar/dv/tests
make clean-all && make run-all-gate

# Run specific test with waveforms
make run-apbx_xbar_2to4-gate-waves

# View waveforms: create_view_cmd() writes a ready-made command
# beside the log; no hand-built gtkwave path needed

# Check documentation
cat projects/components/fabric-gen-ip/lattice-apbx-xbar/PRD.md
cat projects/components/fabric-gen-ip/lattice-apbx-xbar/README.md
```

Remember

1. **Check pre-generated first** - Don't generate unnecessarily
2. **Address map matters** - Always mention BASE_ADDR + 64KB regions
3. **Fair arbitration** - Round-robin per slave
4. **Complete connections** - All APB signals must be wired
5. **Tests available** - 100% passing, comprehensive coverage
6. **Wrappers are TEST SCAFFOLDS** - `rtl/wrappers/` carries only pclk/presetn; instantiate the crossbar directly (see Anti-Pattern 4)
7. **Generator limits** - Up to 16×16 (configurable)

Version: 1.0 **Last Updated:** 2025-10-19 **Maintained By:** RTL Design Sherpa Project